

INTERVIEW PREPARATION GUIDE

Software Engineer Lloyds

Core Java · Spring Boot · Microservices · Testing · SQL
Prepared for Vallu Ravikoti Narendra

How to use this guide. Work the sections in order. The plan on the next page is timed — if you fall behind, drop from the bottom, never from the top.

● **Red dot** = high priority, almost certainly asked. ●● = your differentiator. ● **Amber** = worth knowing.

✓ = self-check before moving on. ⚠ = common trap or something to verify.

Contents

Plan	Lloyds — Software Engineer (Java · Spring Boot · Microservices)	3
01	Core Java and Collections	8
02	Java 8 and Functional Programming	20
03	Spring Boot	27
04	JPA, Hibernate and the N+1 Problem	37
05	Microservices	46
06	Kafka and Messaging	54
07	Spring Security and JWT	59
08	SQL and Query Optimisation	64
09	Coding Round	70
10	Backend System Design	77
11	Talking About Your Experience	82
12	Behavioural Round, Questions to Ask, Logistics	87
13	Testing, Code Review and Engineering Practice	91
14	Final Cheat Sheet — INTERVIEW DAY ONLY	97

Lloyds — Software Engineer (Java · Spring Boot · Microservices)

Interview prep for Vallu Ravikoti Narendra

| How this guide is written

Every topic follows the same four steps:

1. **The context** — what the thing is and what problem it solves, in plain words.
2. **A simple example** — small code you can actually follow.
3. **Why it matters** — where it is used in real work.
4. **"Say this"** — the answer to give in the room.

Every question in this guide has a written answer in the words you would say, and most carry a **real-world example**. The examples are anchored to a **trading and payments platform**, because that is what you worked on — so the analogy you reach for in the room is one you actually understand.

Do not memorise them word for word. Learn the shape, so when the question comes out slightly differently you can still answer it.

| 1. Who you are interviewing with

Lloyds Banking Group is the UK's largest retail and commercial banking group — Lloyds Bank, Halifax, Bank of Scotland and Scottish Widows. They are several years into a large technology transformation, moving off legacy systems towards cloud-hosted, API-driven services, and they have built a substantial engineering organisation in India through the **Lloyds Technology Centre in Hyderabad**. If you are interviewing in India, that is almost certainly the entity.

What that means for you, practically:

- **It is a regulated bank.** Security, auditability, data privacy and change control are not optional extras — they are the job. An answer that includes "and here is how we would know afterwards what happened" lands better here than almost anywhere else.
- **It is a product and platform organisation, not a services company.** They are hiring someone to own and run a service over time, not to be deployed to whichever client needs a body. Answers about maintainability, testing and observability carry real weight.
- **Your domain transfers.** Tradu is a trading platform with onboarding, identity verification and payments across multiple vendors. That is financial services, with KYC and money movement. **Say that early** — domain familiarity is worth a lot to a bank.

⚠ **Verify the format with your recruiter.** What follows is my prediction based on how this kind of role is usually run, not something I can confirm for your specific process. It is a completely normal thing to ask: *"Could you tell me what the rounds look like and what each one focuses on?"* Most recruiters answer that happily, and it costs you nothing.

2. What the interview will probably look like

Round 1 — Technical (45-60 min).

Core Java, collections, Spring Boot, REST, Hibernate/JPA, SQL. Usually one coding question.
→ Files 01, 02, 03, 04, 08, 09.

Round 2 — Technical / design (45-60 min).

Microservices and how services communicate and fail, API design, security, testing, and a deep dive into what you actually built.
→ Files 05, 06, 07, 10, 11, 13.

Round 3 — Behavioural and values (30-45 min).

How you work, how you handle disagreement and mistakes, how you approach quality and risk. UK banks typically run this as a structured competency interview with prepared questions, so **prepared examples matter more here than in a typical Indian services interview.**
→ File 12.

The three things that decide this interview for you

1. Honest calibration. You have 1 year 8 months. Nobody expects architecture ownership. They expect solid fundamentals, clear thinking, and no overclaiming. **The single fastest way to lose this is to claim depth you do not have** — file 05, Q7 and file 11 handle the two places where your resume invites that.

2. Fundamentals, cleanly explained. Collections, Spring Boot, JPA and SQL, explained with the *why*, not just the *what*. That is what a 2-year hire is actually assessed on.

3. Quality practice. JUnit 5, Mockito, SonarQube, code review, logging, root cause analysis — all on your resume, and all things a regulated bank genuinely values. **File 13 is the one most candidates at your level have nothing prepared for**, and it is where you can look better than your years.

3. The plan

I do not know your interview date. Below is a **two-day plan**; if you have only one day, use the compressed version underneath.

Day 1 — fundamentals

Block	Topic	File
0:00 – 0:15	Read this file	README.md

Block	Topic	File
0:15 – 2:00	Core Java and Collections ●● — HashMap above all	01-core-java.md
2:00 – 3:00	Java 8 — streams, lambdas, Optional ●	02-java8-functional.md
3:00 – 3:15	Break	—
3:15 – 4:45	Spring Boot ●●	03-spring-boot.md
4:45 – 6:00	JPA, Hibernate and N+1 ●●	04-jpa-hibernate.md
Evening	Your experience — fill in the templates, say them out loud	11-your-experience.md

Day 2 — the rest, and the parts about you

Block	Topic	File
0:00 – 1:15	Microservices ●● — especially Q7, the honest answer	05-microservices.md
1:15 – 2:00	REST, Spring Security and JWT ●	07-spring-security-jwt.md
2:00 – 2:45	SQL and indexing ●	08-sql-and-databases.md
2:45 – 3:00	Break	—
3:00 – 4:00	Testing, code review and engineering practice ●●	13-engineering-practice.md
4:00 – 5:00	Coding round — type the problems, do not read them	09-coding-round.md
5:00 – 5:45	Behavioural and values ● — say these out loud	12-behavioural-and-values.md
5:45 – 6:15	Skim: messaging, then system design	06 , 10

Interview day

Time	What
45 min	Cheat sheet only. Nothing new.
30 min	Say out loud, twice: your intro, the microservices-depth answer, two project stories
20 min	Re-read the questions you will ask them
20 min	Laptop, charger, network, ID, printed resume, water

Learn nothing new on the day. It only displaces what has settled and raises your anxiety.

If you only have one day

Do, in this order, and stop when you run out: **01 Core Java → 03 Spring Boot → 11 your experience → 05 microservices (Q7 especially) → 13 engineering practice → 04 JPA → 14**

cheat

sheet. Skip 02, 06, 08, 09 and 10 entirely if you must.

Never skip: Core Java, Spring Boot, file 11, and file 05 Q7.

4. The files

File	Topic	Priority
01-core-java.md	OOP, Collections, HashMap internals, String, exceptions	● Highest
02-java8-functional.md	Streams, lambdas, functional interfaces, Optional	● High
03-spring-boot.md	IoC, DI, annotations, bean scopes, auto-configuration	● Highest
04-jpa-hibernate.md	JPA, lazy vs eager, N+1 , transactions, caching	● High
05-microservices.md	Patterns, communication, resilience — read Q7	● Highest
06-kafka-rabbitmq.md	Messaging — you have fundamentals only, answer accordingly	● Medium
07-spring-security-jwt.md	Authentication, authorisation, JWT, API security	● High
08-sql-and-databases.md	Joins, indexing, EXPLAIN, optimisation	● High
09-coding-round.md	Coding problems with full solutions	● High
10-system-design.md	Design framework and worked examples	● Medium
11-your-experience.md	Your Tradu stories, and the four honesty questions	● Highest
12-behavioural-and-values.md	Competency round, questions to ask, logistics	● High
13-engineering-practice.md	Testing, Mockito, code review, SonarQube, CI/CD	● Highest
14-final-cheatsheet.md	One page. Interview morning only.	● Read last

5. Three rules for the room

1. Never say "I don't know" and stop. Say instead:

"I have not used that directly. My understanding is that it does X — is that the direction you mean?"

2. Give the *why*, not just the *what*. "HashMap stores key-value pairs" is a textbook answer. "It gives O(1) lookup because it hashes the key to a bucket instead of scanning" is an engineer's answer. Same length, completely different impression.

3. Name your level before they have to ask. You are 1 year 8 months in. Saying "here is what I have done, and here is where my depth ends" early makes everything else you say more credible. It is the opposite of a weakness in an interviewer's eyes.

01 · Core Java and Collections

Time needed: 105 minutes

This is where a bank screens hardest for someone at your level. At under two years, the technical round is mostly fundamentals — because fundamentals are what predict whether you will write reliable code in an unfamiliar codebase. Expect rapid-fire questions here.

If you only master one topic in this file, make it HashMap. It is the single most asked Java interview question in India.

Part A · OOP — the four pillars

You will be asked to name them and give an example of each. Have a real example ready, not a "Dog extends Animal" one.

| Q1. Encapsulation

What it is: keeping data private and exposing it only through methods.

```
public class Account {  
    private double balance;                // nobody can touch this directly  
  
    public void deposit(double amount) {  
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");  
        this.balance += amount;           // the rule is enforced here  
    }  
  
    public double getBalance() { return balance; }  
}
```

Why it matters: if `balance` were public, any code anywhere could set it to a negative number and no validation would run. Encapsulation means the object protects its own rules. In a banking system that is not a style preference.

Say this.

"Encapsulation means the object owns its data and its rules. The fields are private and the only way in is through methods that validate. On a trading platform an account balance cannot simply be assigned — it has to go through a method that checks the transaction is valid and the account is in a state that permits it. If that field were public, any code anywhere could set it and skip every check, and you would never find out where it happened."

Q2. Inheritance

One class reuses another's behaviour with `extends`. A `SavingsAccount extends Account` gets `deposit()` for free and adds interest calculation.

The follow-up: "Why does Java not support multiple inheritance of classes?"

"Because of the diamond problem. If B and C both extend A and override the same method, and D extends both, the compiler cannot decide which version D inherits. Java avoids that by allowing only one superclass. You can implement multiple interfaces, and since Java 8 interfaces can have default methods — so if two interfaces give the same default method, the compiler forces you to override it and resolve the conflict explicitly."

Q3. Polymorphism •

"One interface, many implementations." Two kinds, and interviewers want both.

	Overloading (compile-time)	Overriding (runtime)
What changes	Same method name, different parameters	Same signature, different class
Decided	At compile time, by the compiler	At runtime, by the actual object
Also called	Static polymorphism	Dynamic polymorphism

```
// Overloading - same name, different parameter lists
void notify(String email) { }
void notify(String email, String sms) { }

// Overriding - the subclass replaces the parent's behaviour
class Notifier { void send() { System.out.println("generic"); } }
class EmailNotifier extends Notifier { @Override void send() { System.out.println("email"); } }

Notifier n = new EmailNotifier();
n.send(); // "email" - decided by the OBJECT, not the reference type
```

That last line is the whole point of runtime polymorphism, and it is what makes Spring's dependency injection work: the field is declared as an interface, and the concrete implementation is chosen at runtime.

Real-time example. In the notification flow, the service holds a `List<Notifier>` containing an email notifier and an SMS notifier. It calls `notifier.send(...)` on each without knowing or caring which is which. When a push notifier is added later, the sending code does not change at all — that is polymorphism doing real work.

Trap: "Can you override a static method?" → No. Static methods belong to the class, not the instance, so they are **hidden**, not overridden. Which one runs is decided by the reference type, not the object.

Say this.

"Overloading is compile time — same method name, different parameters, and the compiler picks. Overriding is runtime — a subclass replaces the parent's behaviour, and which one runs depends on the actual object, not the reference type.

Overriding is what makes dependency injection work. My service holds a `Notifier` reference, and at runtime it is an email or SMS implementation. The service never changes when a new channel is added."

Q4. Abstraction

Hiding *how* something works and exposing *what* it does.

● Abstract class vs interface — near-certain question

	Abstract class	Interface
Fields	Can have normal instance fields	Only <code>public static final</code> constants
Constructor	Yes	No
Methods	Abstract and concrete	Abstract, plus <code>default</code> and <code>static</code> since Java 8
Inheritance	One only	Many
Access modifiers	Any	Methods are public

When to use which — say it this way:

"An abstract class is for an 'is-a' relationship where the subclasses share state and some common implementation — a base `Transaction` class holding amount and timestamp. An interface is a contract about capability with no shared state, like `Auditable` or `PaymentProcessor`. Since I can implement many interfaces but extend only one class, I default to interfaces and use an abstract class when there is real shared code and fields."

Real-time example — say this if they ask for one.

"On the trading platform we integrated **multiple payment vendors**, and that is the natural interface. `PaymentVendor` defines the contract — initiate, verify status, refund — and each vendor implements it completely differently. There is no shared code to inherit, only a shared contract, so the service depends on the interface and Spring injects the right implementation. Adding a vendor does not change the calling code.

An **abstract class** would be right where implementations genuinely share state and behaviour — a base transaction type holding the amount, currency, timestamp and a common `validate()` that all of them run. That is shared code, which an interface cannot give me."

Part B · Collections ●

Q5. The map of the framework

Collection (interface)

└ List	– ordered, allows duplicates	→ ArrayList, LinkedList, Vector
└ Set	– no duplicates	→ HashSet, LinkedHashSet, TreeSet
└ Queue	– FIFO processing	→ PriorityQueue, ArrayDeque

Map (separate, NOT a Collection) – key/value → HashMap, LinkedHashMap, TreeMap, Hashtable

⚠ `Map` does **not** extend `Collection`. It stores pairs, not single elements. A small detail interviewers like to check.

Say this — how you choose a collection.

"I pick by the access pattern. If I need order and index access, `List`, and `ArrayList` unless I have a reason. If I need uniqueness, `Set`. If I need lookup by a key, `Map`. Then the second question is whether ordering matters — `LinkedHashSet` or `TreeSet` instead of `HashSet` — and whether it is accessed by multiple threads, which means `ConcurrentHashMap` rather than `HashMap`."

Real-time example. In a payment reconciliation job: a `List` for the page of transactions being processed, a `Set` of already-processed reference numbers so a retry does not process the same payment twice, and a `Map` of status code to description loaded once instead of looked up per row.

Q6. ArrayList vs LinkedList ●

	ArrayList	LinkedList
Structure	Growable array	Doubly linked list
Get by index	O(1)	O(n) — must walk the chain
Add/remove at end	O(1) amortised	O(1)
Add/remove in middle	O(n) — shifts elements	O(1) <i>once you are at the node</i>
Memory	Less	More — each node stores two pointers

Say this:

"I use ArrayList by default. Random access is O(1) and the elements sit together in memory, which is cache friendly. LinkedList only wins when I am inserting or removing frequently in the middle while already holding a reference to that position. In practice that is rare, and even for a queue I would use ArrayDeque."

Real-time example. A trade history endpoint loads a page of transactions and iterates them to build the response — read-heavy, index access, no insertion in the middle. That is an `ArrayList`. I have not yet hit a case in real work where `LinkedList` was the right answer.

How does ArrayList grow? It starts at capacity 10, and when full it creates a new array about **1.5 times** the size and copies everything across. That copy is why `new ArrayList<>(expectedSize)` matters when you know the size up front.

Q7. ●● How does HashMap work internally?

Learn this properly. It is the most asked Java question in Indian interviews.

Storing a value — `map.put(key, value)`

1. Java calls `key.hashCode()` to get an int.
2. HashMap applies its own **hash spreading** on top: `h ^ (h >>> 16)`. This mixes the high bits into the low bits, because the next step only uses the low bits.
3. It finds the bucket: `index = (n - 1) & hash`, where `n` is the array length. Since `n` is always a power of two, this is a fast bitwise AND instead of a modulus.
4. If the bucket is empty, it stores the entry there.
5. If the bucket already has entries — a **collision** — it walks them comparing with `equals()`. Same key means replace the value; otherwise append the new entry.

The structure

Internally it is an **array of buckets**, and each bucket holds a **linked list** of entries. Since **Java 8**, once a single bucket exceeds **8 entries** (and the table is at least 64 long) that list is converted into a **balanced red-black tree**, turning worst-case lookup from O(n) into O(log n). Below 6 entries it converts back.

Resizing

Default capacity **16**, load factor **0.75**. When size passes $16 \times 0.75 = 12$, the array doubles to 32 and everything is **rehashed** into new buckets. Resizing is expensive, which is why you presize a map you know will be large.

Complexity

O(1) average for get and put. O(log n) worst case since Java 8, thanks to treeification.

Real-time example — where you actually used one

"Two places. Reference data — instrument codes, currency codes, status descriptions — loaded once into a `HashMap` keyed by code, so a loop over transactions looks each one up in $O(1)$ instead of hitting the database per row. And for idempotency: a map of already-processed request or message IDs, checked before acting, so a duplicate or retried request becomes a no-op instead of a second payment."

• The follow-up that catches people: equals and hashCode

"If you use a custom object as a key you must override both `hashCode()` and `equals()`. The contract is that **equal objects must have equal hash codes**. If I override `equals` but not `hashCode`, two objects that are equal can produce different hash codes, land in different buckets, and the map will never find the entry I stored. That is also why `HashMap` keys should be **immutable** — if a field used in the hash changes after insertion, the entry is effectively lost, because it is sitting in a bucket that no longer matches its hash."

Q8. HashMap vs Hashtable vs ConcurrentHashMap •

	HashMap	Hashtable	ConcurrentHashMap
Thread safe	No	Yes	Yes
How	—	Locks the whole map	Locks only the affected bucket
Null key/value	One null key, many null values	Neither	Neither
Performance	Fastest single-threaded	Slow — everything serialises	Fast under concurrency
Status	Standard	Legacy, do not use	Standard for concurrent use

"Hashtable synchronises every method, so all threads queue behind one lock. That is why it is effectively obsolete. `ConcurrentHashMap` locks at bucket level, so threads writing to different buckets do not block each other, which is why it scales."

Real-time example. A Spring `@Service` is a **singleton**, so any collection held in a field is shared by every request thread at once. If I keep an in-memory counter or cache in a service — say a per-branch request count — a plain `HashMap` will corrupt under load and can even spin forever on an older JDK. That field has to be a `ConcurrentHashMap`.

Q9. HashSet vs LinkedHashSet vs TreeSet

	Ordering	Backing structure	Complexity
HashSet	None guaranteed	HashMap	O(1)
LinkedHashSet	Insertion order	HashMap + linked list	O(1)
TreeSet	Sorted	Red-black tree	O(log n)

Useful detail: `HashSet` is literally a `HashMap` where every value is the same dummy object. That is why its rules about `hashCode` and `equals` are identical.

Real-time example. `HashSet` for "which account numbers have already been processed in this batch" — I only care about membership. `TreeSet` when the order matters, like interest rate slabs that must be walked from lowest to highest to find the applicable band. `LinkedHashSet` when I need uniqueness but the output must stay in the order the records arrived, which matters for a report a human reads.

Say this.

"All three guarantee uniqueness; the difference is ordering and cost. `HashSet` is O(1) with no order guarantee. `LinkedHashSet` is also O(1) but keeps insertion order, which matters when a human reads the output. `TreeSet` keeps them sorted, which costs O(log n) because it is a red-black tree. I use `HashSet` by default and only pay for ordering when I actually need it."

Q10. Fail-fast vs fail-safe iterators

"Fail-fast iterators, like `ArrayList`'s and `HashMap`'s, throw `ConcurrentModificationException` if the collection is structurally modified while iterating. They track a `modCount` and check it on each `next()`. Fail-safe iterators, like `ConcurrentHashMap`'s or `CopyOnWriteArrayList`'s, work on a snapshot or allow concurrent access, so they do not throw — but they may not reflect the very latest changes."

How do you remove safely while iterating? Use `iterator.remove()`, or `collection.removeIf(predicate)`. Never `list.remove()` inside a for-each loop.

Q11. Comparable vs Comparator ●

```
// Comparable - the class defines its OWN natural ordering. One only.
class Txn implements Comparable<Txn> {
    public int compareTo(Txn other) { return this.date.compareTo(other.date); }
}

// Comparator - ordering defined OUTSIDE the class. As many as you like.
Comparator<Txn> byAmount = Comparator.comparing(Txn::getAmount);
list.sort(byAmount.reversed());
```

"Comparable is the natural order and lives inside the class, so there can be only one. Comparator lives outside, so I can have several — sort transactions by date on one screen and by amount on another without touching the entity."

Real-time example. `Transaction` implements `Comparable` on `createdAt`, because newest-first is its natural order everywhere in the application. But a trade history screen lets the user sort by amount, by status or by instrument — three `Comparator`s, none of which required changing the entity.

Part C · Strings, immutability and memory

Q12. Why is String immutable? ●

Once created, a String's value can never change. `s.concat("x")` returns a **new** String.

The four reasons — give at least two:

1. **String pool.** Java reuses identical literals to save memory. That is only safe if nobody can change them underneath other references.
2. **Security.** Usernames, file paths and DB URLs are Strings. If they were mutable, a value could be validated and then changed before use.
3. **Thread safety.** Immutable objects can be shared across threads with no locking.
4. **HashCode caching.** String caches its hash, which makes it a fast HashMap key. That is only valid if the value cannot change.

Real-time example. Account numbers and transaction reference numbers are Strings used as map keys and passed between services. If a String were mutable, code holding a reference could change an account number *after* it had been validated and after it had been used to place an entry in a map — the entry would then be unreachable. Immutability is what makes them safe to share.

Say this.

"Because too much depends on a String not changing under you. The string pool reuses identical literals, which is only safe if nobody can modify one. Strings are used for account numbers, file paths and connection URLs, so if they were mutable a value could be validated and then changed before it was used. And String caches its own hash code, which is what makes it a fast HashMap key — that cache is only valid because the value is fixed."

Q13. String vs StringBuilder vs StringBuffer

	Mutable?	Thread safe	Use when
String	No	Yes (immutable)	Fixed values
StringBuilder	Yes	No	Building strings — the default
StringBuffer	Yes	Yes (synchronised)	Legacy; rarely needed

"Concatenating in a loop with `+` creates a new String every iteration, so a thousand-iteration loop creates a thousand throwaway objects. StringBuilder mutates one buffer, so it is $O(n)$ instead of $O(n^2)$."

Q14. `==` vs `.equals()`

`==` compares **references** — are these the same object in memory.

`.equals()` compares **values** — as defined by the class.

```
String a = "hello";           // goes into the string pool
String b = "hello";           // reuses the SAME pooled object
String c = new String("hello"); // forces a NEW object on the heap

a == b      // true  - same pooled reference
a == c      // false - different objects
a.equals(c) // true  - same characters
```

This is a favourite quick question. Learn the three lines.

Say this.

"`==` compares references — is this the same object in memory. `.equals()` compares values, as the class defines them. For Strings this catches people out because literals are pooled, so two identical literals are the same object and `==` happens to be true. But `new String("abc")` forces a separate object and `==` is false, while `.equals()` is still true. So I never use `==` for value comparison — it works by accident until it does not."

Q15. Stack vs Heap

- **Stack** — one per thread. Holds method calls, local variables and references. Freed automatically when the method returns.
- **Heap** — shared by all threads. Holds all objects. Managed by the garbage collector.

`StackOverflowError` means runaway recursion. `OutOfMemoryError` means the heap is full, usually a leak or an unbounded collection.

Say this.

"The stack is per thread and holds method frames, local variables and references. It is cleaned up automatically when a method returns. The heap is shared across threads and holds all the objects, and the garbage collector manages it. A `StackOverflowError` means runaway recursion; an `OutOfMemoryError` means the heap filled up, which in my experience usually means something unbounded — loading an entire table into a list instead of paginating it."

Part D · Exceptions

Q16. Checked vs unchecked ●

```
Throwable
├── Error                – JVM level, do not catch (OutOfMemoryError)
└── Exception
    ├── RuntimeException – UNCHECKED (NullPointerException, IllegalArgumentException)
    └── everything else – CHECKED (IOException, SQLException)
```

- **Checked:** the compiler forces you to catch it or declare `throws`. Represents recoverable, expected conditions — a missing file, a network failure.
- **Unchecked:** extends `RuntimeException`. Usually a programming bug. No compiler enforcement.

Say this:

"In application code I mostly use unchecked exceptions and a custom hierarchy — a `TransactionNotFoundException` extending `RuntimeException` — because forcing every caller to catch and rethrow adds noise. I handle them centrally in Spring with `@ControllerAdvice` so the API returns a consistent error shape. It also matters for `@Transactional`, because by default Spring only rolls back on unchecked exceptions."

That last sentence connects two topics and sounds like real experience. Use it.

Real-time example. On Tradu the backend served a React frontend, so a consistent error contract mattered a lot — the frontend team needed one shape to handle. Custom unchecked

exceptions such as `TransactionNotFoundException` and an invalid-state exception, both extending a common `BusinessException`, mapped through `@RestControllerAdvice` to 404 and 409. Unchecked also matters for `@Transactional`, because a checked exception would **commit** instead of rolling back.

Q17. try-with-resources and finally

```
try (Connection conn = dataSource.getConnection()) {  
    // conn.close() is called automatically, even if an exception is thrown  
}
```

Anything implementing `AutoCloseable` works. This replaced the old `finally { if (conn != null) conn.close(); }` pattern.

Trap: "Does `finally` always run?" → Almost always. Not if the JVM exits (`System.exit()`) or the thread is killed. And if `finally` contains a `return`, it overrides the one in `try` — which is why you never return from `finally`.

Say this.

"Try-with-resources closes anything that implements `AutoCloseable` automatically, even if an exception is thrown, so I do not need a `finally` block that null-checks and closes. It replaced a pattern that people got wrong constantly — forgetting to close in one branch, or throwing a second exception inside `finally` that hid the original one. With JPA and Spring I rarely manage connections myself, but for file and stream handling it is what I use."

Q18. `throw` VS `throws` VS `final` VS `finally` VS `finalize`

- `throw` — actually raise an exception now.
- `throws` — declare that this method may raise one.
- `final` — a constant, a non-overridable method, or a non-extendable class.
- `finally` — the block that runs whatever happens.
- `finalize()` — a deprecated, unreliable method the GC used to call. Never use it.

Say this.

"`throw` raises an exception now, `throws` declares that a method might. `final` means a constant, a method that cannot be overridden or a class that cannot be extended. `finally` is the block that runs whatever happens. And `finalize()` is a deprecated method the garbage collector used to call — it is unreliable and should not be used; try-with-resources or an explicit close is the correct approach."

✓ Check yourself before moving on

1. Explain **HashMap put and get end to end** — hashing, buckets, collisions, treeification, resizing. Out loud, without notes.
2. Explain the `equals` / `hashCode` contract and what breaks if you ignore it.
3. Overloading vs overriding, with an example of each.
4. Abstract class vs interface, and when you would pick each.
5. Explain the three lines of the `==` vs `.equals()` String example.

02 · Java 8 and Functional Programming

Time needed: 60 minutes

Your resume says "Java 8+", so this **will** be tested. Interviewers usually ask you to explain streams and then write one on the spot.

First, the context: what problem did Java 8 solve?

Before Java 8, processing a collection meant writing *how* to loop:

```
// imperative - you describe every step
List<String> names = new ArrayList<>();
for (Txn t : txns) {
    if (t.getAmount() > 1000) {
        names.add(t.getCustomerName().toUpperCase());
    }
}
Collections.sort(names);
```

Java 8 let you describe *what* you want instead:

```
// declarative - the intent is visible in one read
List<String> names = txns.stream()
    .filter(t -> t.getAmount() > 1000)
    .map(t -> t.getCustomerName().toUpperCase())
    .sorted()
    .collect(Collectors.toList());
```

Same result, but the second reads as a sentence: filter, map, sort, collect. That is the point — less boilerplate and clearer intent.

Q1. What is a lambda? ●

A short way to write an implementation of an interface that has exactly one method.

```
// before Java 8
Runnable r = new Runnable() {
    public void run() { System.out.println("running"); }
};

// with a lambda
Runnable r = () -> System.out.println("running");
```

The compiler knows `Runnable` has one method, so you only supply the body.

Say this.

"A lambda is a compact way to supply the implementation of an interface that has one method. Before Java 8 I would write an anonymous inner class of five lines to pass behaviour into a method; a lambda is the same thing with the boilerplate removed. It is what makes the Streams API readable — `filter(t -> t.getAmount() > 1000)` says what it does in one line."

Q2. What is a functional interface? •

An interface with **exactly one abstract method**. That is what makes it a valid target for a lambda. `@FunctionalInterface` is optional but makes the compiler enforce it.

The four you must know:

Interface	Method	Takes	Returns	Used by
<code>Predicate<T></code>	<code>test(T)</code>	T	boolean	<code>filter</code>
<code>Function<T,R></code>	<code>apply(T)</code>	T	R	<code>map</code>
<code>Consumer<T></code>	<code>accept(T)</code>	T	void	<code>forEach</code>
<code>Supplier<T></code>	<code>get()</code>	nothing	T	<code>orElseGet</code> , lazy values

```
Predicate<Txn> isLarge = t -> t.getAmount() > 1000;
Function<Txn, String> name = t -> t.getCustomerName();
Consumer<Txn> log = t -> System.out.println(t);
Supplier<Txn> empty = () -> new Txn();
```

Also worth naming: `BiFunction<T,U,R>` (two arguments), `UnaryOperator<T>` (T to T), and `BinaryOperator<T>` (two Ts to one T, used by `reduce`).

Say this.

"An interface with exactly one abstract method. That single method is what lets the compiler match a lambda to it. `@FunctionalInterface` is optional but I add it, because it makes the compiler fail if someone later adds a second abstract method and silently breaks every lambda using it. The four I use most are `Predicate` for filtering, `Function` for mapping, `Consumer` for side effects and `Supplier` for a lazily produced value."

Q3. Method references

Shorthand for a lambda that just calls one method.

```
.map(t -> t.getCustomerName()) → .map(Txn::getCustomerName)
.forEach(x -> System.out.println(x)) → .forEach(System.out::println)
```

Say this.

"A method reference is a shorter form of a lambda that does nothing but call one method. `t -> t.getCustomerName()` becomes `Txn::getCustomerName`. It is purely readability — the compiler produces the same thing — but in a stream chain it removes noise and leaves only the intent."

Q4. ●● Streams — intermediate vs terminal operations

This distinction is asked constantly.

Intermediate operations return another Stream and are **lazy** — they do nothing until a terminal operation runs. `filter`, `map`, `flatMap`, `sorted`, `distinct`, `limit`, `skip`, `peek`.

Terminal operations produce a result and **trigger** the whole pipeline. `collect`, `forEach`, `reduce`, `count`, `min`, `max`, `anyMatch`, `allMatch`, `findFirst`.

```
txns.stream().filter(t -> t.getAmount() > 1000); // NOTHING happens - no terminal op
```

Why laziness matters — the answer that impresses:

"Because intermediate operations are lazy, the stream can fuse them into a single pass and stop early. `stream().filter(...).findFirst()` does not filter the whole list and then take the first — it stops at the first match. With `limit(10)` on a million records, only enough elements are processed to produce ten."

A stream can only be consumed once. Reusing one throws `IllegalStateException: stream has already been operated upon or closed`.

Q5. The stream methods you must be able to write ●

```
// filter + map + collect
List<String> big = txns.stream()
    .filter(t -> t.getAmount() > 1000)
    .map(Txn::getCustomerName)
    .collect(Collectors.toList());

// sum with reduce, and the shorter specialised form
double total = txns.stream().mapToDouble(Txn::getAmount).sum();
double same = txns.stream().map(Txn::getAmount).reduce(0.0, Double::sum);

// GROUP BY - very commonly asked
Map<String, List<Txn>> byStatus =
    txns.stream().collect(Collectors.groupingBy(Txn::getStatus));

// group and count
Map<String, Long> countByStatus =
    txns.stream().collect(Collectors.groupingBy(Txn::getStatus, Collectors.counting()));

// group and sum
Map<String, Double> totalByStatus = txns.stream()
    .collect(Collectors.groupingBy(Txn::getStatus,
        Collectors.summingDouble(Txn::getAmount)));

// sort by a field, descending
txns.stream().sorted(Comparator.comparing(Txn::getAmount).reversed())

// flatMap - flatten nested collections into one stream
List<Item> all = orders.stream()
    .flatMap(o -> o.getItems().stream())
    .collect(Collectors.toList());
```

Real-time example — where this replaced real code. The report summary needed totals per status: how many transactions are pending, settled and failed, and the total value of each. Written as nested loops with counters and a map built by hand, that is about twenty lines. One `groupingBy` with `summingDouble` replaces it, and the intent is visible in a single read.

⚠ But note the boundary: **do this in SQL when the data is in the database**. Streaming a million rows into memory to group them is much slower than a `GROUP BY`. Streams are for data you already have in hand.

That caveat is worth saying out loud — it shows you know where the work should happen.

● `map` vs `flatMap` is a guaranteed question:

"`map` transforms one element into one element. `flatMap` transforms one element into a **stream** of elements and then flattens them all into a single stream. If each order has a list of items, `map` gives me a stream of lists, and `flatMap` gives me a stream of items."

Real-time example. An order has a list of executions, or an onboarding case has a list of verification checks. To get every check across a page of cases, `map` gives me a stream of *lists* of checks; `flatMap` gives me a single stream of checks I can filter and count.

Q6. Optional ●

Introduced to make "this might be absent" explicit in the type, instead of returning `null` and hoping the caller checks.

```
Optional<Txn> found = repository.findById(id);

// good
String name = found.map(Txn::getCustomerName).orElse("Unknown");
found.ifPresent(t -> log.info("Found {} ", t.getId()));
Txn txn = found.orElseThrow(() -> new TransactionNotFoundException(id));

// bad - this is just a null check with extra steps
if (found.isPresent()) { Txn t = found.get(); }
```

"Optional makes the possibility of absence part of the method signature, so the caller cannot ignore it by accident. I use `map`, `orElse` and `orElseThrow` rather than `isPresent` and `get`, because that defeats the purpose. Spring Data returns `Optional` from `findById` for exactly this reason."

Real-time example. `txnRepository.findById(id)` returns `Optional<Transaction>`. In a service that becomes

`.orElseThrow(() -> new TransactionNotFoundException(id))`, which the `@RestControllerAdvice` turns into a clean 404. Before `Optional`, that same code path returned `null` and produced a `NullPointerException` and a 500 — which tells the caller nothing useful.

orElse vs orElseGet — a good detail: `orElse(buildDefault())` evaluates the argument **always**, even when a value is present. `orElseGet(() -> buildDefault())` only runs it when needed. It matters when the default is expensive.

Q7. Parallel streams — and when not to use them

```
list.parallelStream().filter(...).collect(...);
```

"It splits the work across the fork-join pool. It only helps with a large dataset and a genuinely CPU-bound operation with no shared mutable state. For small collections the coordination overhead makes it slower, and for I/O-bound work like database calls it is the wrong tool. I would measure before using it, not assume."

Q8. Other Java 8 features worth naming

- **Default and static methods in interfaces** — let you add a method to an interface without breaking every existing implementation.
- **java.time** — `LocalDate`, `LocalDateTime`, `Duration`. Immutable and thread safe, unlike the old `Date` and `Calendar`.
- **CompletableFuture** — asynchronous composition, useful for calling several services in parallel.

Say this.

"The three I actually use beyond streams. **Default methods** in interfaces, which let a library add a method without breaking every existing implementation — that is how `Collection` gained `stream()` without breaking the world. **java.time**, because the old `Date` and `Calendar` were mutable and not thread safe, and `LocalDate` and `LocalDateTime` are neither. And **CompletableFuture** when I need to call two independent services in parallel instead of one after the other."

Q9. Later versions, in one line each

If asked "what have you used beyond Java 8":

- **Java 11** — `var` for local variables, `String.isBlank()`, `strip()`, `lines()`.
- **Java 17 (LTS)** — records, sealed classes, switch expressions, text blocks.
- A **record** is a compact immutable data carrier: `record TxnDto(String id, double amount) {}` generates the constructor, getters, `equals`, `hashCode` and `toString`. Very useful for DTOs.

Say this — and be honest about what you have used.

"My day-to-day is Java 8, which is still what most banking codebases run. I have kept up with the LTS releases: Java 11 added `var` and some String utilities, and Java 17 added records, sealed classes and switch expressions. Records are the one I would most want to use — a DTO becomes a single line instead of forty lines of constructor, getters, `equals` and `hashCode`."

Do not claim production experience with a version you have not used. Knowing what changed and why is enough at this level.

✓ Check yourself before moving on

1. Write a stream that groups transactions by status and sums the amount in each group. From memory.
2. Explain intermediate vs terminal, and why laziness lets a stream stop early.
3. Explain `map` vs `flatMap` in two sentences.
4. Name the four core functional interfaces and which stream method uses each.

03 · Spring Boot

Time needed: 90 minutes

"Java → Spring Boot" is listed as the **primary skill** on the job description. Along with Core Java, this is where most of your technical round will be spent.

| First, the context: what problem does Spring solve?

The problem: tight coupling

Without Spring, an object creates the things it depends on:

```
public class TransactionService {  
    private final EmailNotifier notifier = new EmailNotifier();    // hard-wired  
}
```

Two problems. To switch to SMS you must edit this class. And in a unit test you cannot replace the notifier with a fake, so your test sends real emails.

The solution: Inversion of Control

The object stops creating its dependencies and **receives** them instead:

```
@Service  
public class TransactionService {  
    private final Notifier notifier;    // an interface  
  
    public TransactionService(Notifier notifier) {    // handed in from outside  
        this.notifier = notifier;  
    }  
}
```

Now the class depends on an interface, not a concrete class. Something else decides which implementation to supply. That "something else" is the Spring container.

Inversion of Control (IoC) is the principle: control of creating objects is inverted, moving from your class to the framework.

Dependency Injection (DI) is how Spring implements it: it constructs objects and passes their dependencies in.

Say it in one line:

"IoC is the principle that the framework, not my class, controls object creation and wiring. Dependency injection is the mechanism. The practical benefit is that my classes depend on interfaces, so I can swap implementations and inject mocks in tests without changing the class."

Real-time example. The card approval service depends on a `NotificationSender` interface. In production Spring injects the RabbitMQ implementation. In unit tests I pass a mock and assert that approval publishes exactly one message — no broker, no emails, and the test runs in milliseconds. That is only possible because the service never says `new`.

And Spring Boot on top of Spring

Plain Spring needed a lot of XML and manual configuration. Spring Boot adds:

- **Auto-configuration** — sees a database driver on the classpath and configures a `DataSource` for you.
- **Starter dependencies** — `spring-boot-starter-web` pulls in a compatible set of libraries so you are not resolving versions by hand.
- **Embedded server** — Tomcat is inside the JAR, so `java -jar app.jar` runs it. No separate application server to install.
- **Production features** — Actuator gives health checks and metrics endpoints.

"Spring Boot is Spring with opinionated defaults. It removes the configuration work so you start with a running application and override only what you need."

Q1. • The three types of injection, and which to use

```
// 1. CONSTRUCTOR injection - what you should use
@Service
public class TxnService {
    private final TxnRepository repo;
    public TxnService(TxnRepository repo) { this.repo = repo; }
}

// 2. FIELD injection - convenient, but avoid
@Autowired private TxnRepository repo;

// 3. SETTER injection - for genuinely optional dependencies
@Autowired public void setRepo(TxnRepository repo) { this.repo = repo; }
```

Why constructor injection — four reasons, give two or three:

1. The field can be `final`, so the object is **immutable** once built.
2. Dependencies **cannot be missing** — the object cannot be constructed without them.
3. **Testable without Spring** — `new TxnService(mockRepo)` just works. With field injection you need reflection or a Spring context.

4. It makes bad design **visible** — a constructor with eight parameters is obviously doing too much. Field injection hides that.

Since Spring 4.3, if a class has one constructor you do not even need `@Autowired`.

Say this.

"I use constructor injection. Three reasons. The fields can be `final`, so the object is immutable once it is built. The dependencies cannot be missing, because you cannot construct the object without them — with field injection you find out at runtime with a null pointer. And it is testable without Spring at all: `new TxnService(mockRepo)` just works, whereas field injection needs reflection or a full context.

There is a fourth benefit — it makes bad design visible. A constructor with eight parameters obviously does too much. Field injection hides that."

Q2. • The annotations you must know

Annotation	What it does
<code>@SpringBootApplication</code>	The main class. Combines the next three.
<code>@Configuration</code>	This class defines beans
<code>@EnableAutoConfiguration</code>	Turn on Boot's auto-config
<code>@ComponentScan</code>	Scan this package and below for components
<code>@Component</code>	A generic Spring-managed bean
<code>@Service</code>	A <code>@Component</code> marking business logic
<code>@Repository</code>	A <code>@Component</code> for data access — also translates DB exceptions into Spring's <code>DataAccessException</code> hierarchy
<code>@Controller</code>	Returns view names
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> — returns JSON
<code>@Autowired</code>	Inject a dependency
<code>@Qualifier("name")</code>	Choose between multiple beans of the same type
<code>@Primary</code>	Make one bean the default when several match
<code>@Value("\${prop}")</code>	Inject a property from configuration
<code>@Bean</code>	Declare a bean from a method, usually for third-party classes
<code>@Transactional</code>	Wrap the method in a database transaction
<code>@ControllerAdvice</code>	Global exception handling across controllers

Trap: '@Component vs @Service vs @Repository — is there a functional difference?'

"Technically @Service and @Component behave identically; the difference is intent, so the layer is obvious to a reader and to tooling. @Repository is the exception — it does real work, translating vendor-specific database exceptions into Spring's `DataAccessException` hierarchy, so my service layer is not coupled to a particular database's error codes."

Q3. REST annotations

```
@RestController
@RequestMapping("/api/transactions")
public class TxnController {

    @GetMapping("/{id}")
    public TxnDto get(@PathVariable Long id) { ... }

    @GetMapping
    public List<TxnDto> search(@RequestParam(defaultValue = "0") int page) { ... }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public TxnDto create(@Valid @RequestBody CreateTxnRequest req) { ... }

    @PutMapping("/{id}")
    public TxnDto replace(@PathVariable Long id, @RequestBody TxnDto dto) { ... }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) { ... }
}
```

@PathVariable reads from the URL path. @RequestParam reads a query parameter.
@RequestBody deserialises the JSON body. @Valid triggers Bean Validation.

Say this.

"@RestController is @Controller plus @ResponseBody, so the return value is serialised to JSON rather than resolved as a view name. @RequestMapping at class level sets the base path. Then @PathVariable reads a value out of the URL path, @RequestParam reads a query parameter, and @RequestBody deserialises the JSON body into an object. I put @Valid on the request body so Bean Validation runs before the method is entered, which means invalid input never reaches my business logic."

Q4. • Bean scopes

Scope	Meaning
singleton	The default. One instance for the whole application context.
prototype	A new instance every time it is requested.
request	One per HTTP request (web only).
session	One per HTTP session (web only).

• **The follow-up:** *"Singleton beans are shared across all requests. Is that a problem?"*

"Only if the bean holds mutable state. Spring beans should be stateless — request data belongs in method parameters and local variables, not in instance fields. A singleton service with a mutable instance field is a genuine concurrency bug, because many threads share that one object."

Real-time example — a real bug this causes. If a service stores the current request's user in an instance field to avoid passing it around, two concurrent approvals will overwrite each other's value, and an action gets attributed to the wrong user. On a regulated financial platform that is an audit failure, not just a bug. Request-scoped data belongs in method parameters, or in the `SecurityContext`, never in a field on a singleton.

That answer shows you understand *why* the default is safe, which is what they are checking.

Q5. • Global exception handling

Nearly always asked, because it separates people who have shipped an API from people who have followed a tutorial.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(TransactionNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(TransactionNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("TXN_NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(new ErrorResponse("VALIDATION_ERROR", msg));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception ex) {
        log.error("Unexpected error", ex);    // log the detail...
        return ResponseEntity.status(500)
            .body(new ErrorResponse("INTERNAL_ERROR", "Something went wrong"));
        // ...but do not leak it to the client
    }
}

```

Say why it matters:

"It keeps error handling out of every controller, so the API returns one consistent error shape. The last handler matters for security: I log the stack trace but return a generic message, because a raw stack trace tells an attacker about your framework versions and internal structure. On a banking system that is a real concern."

Real-time example. Before this was centralised, each controller had its own try/catch and every endpoint returned a slightly different error body — some with a `message`, some with an `error`, some just a stack trace. The UI team had to special-case each one. One `@RestControllerAdvice` gave every endpoint the same `{ code, message, timestamp }` shape, and their error handling became a single function.

Q6. Bean Validation

```
public class CreateTxnRequest {
    @NotBlank(message = "Account number is required")
    private String accountNumber;

    @NotNull @DecimalMin(value = "0.01", message = "Amount must be positive")
    private BigDecimal amount;

    @Email private String notifyEmail;
}
```

`@Valid` on the `@RequestBody` triggers it, and a failure raises `MethodArgumentNotValidException`, handled above.

⚠ Note `BigDecimal` for money, never `double`. Floating point cannot represent decimal fractions exactly, so `0.1 + 0.2` is not `0.3`. In a banking interview, using `BigDecimal` for currency is a detail that gets noticed.

Say this.

"Bean Validation with `@Valid` moves input checking to the edge of the application, so the controller rejects bad requests before any business code runs. The rules live on the DTO next to the field, which is easier to keep correct than validation scattered through a service.

The important caveat is that this is not a security control. It runs on my server, so it is trustworthy, but it is only one layer — for something like a transfer amount I would still check business rules in the service, because a validation annotation cannot know the account balance."

Real-time example. On the card request form the DTO carried `@NotBlank` on the employee ID and `@DecimalMin` on the limit. Those failures come back as a 400 with a field-level message, handled by the same `@RestControllerAdvice` as everything else, so the UI could show the error against the right field.

Q7. Configuration and profiles

```
**Say this.**
> "Profiles let one build behave differently per environment, so the artefact that passed
> SIT is the exact artefact that goes to UAT and production – only the profile changes.
> Anything environment-specific, especially credentials, comes from environment variables
> rather than being committed. And I use `@ConfigurationProperties` rather than scattering
> `@Value` annotations, because it groups related settings into one typed object that
> fails at startup if something is missing, instead of failing later at runtime."

# application.yml
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}    # from environment, never committed
  ---
spring:
  config.activate.on-profile: dev
  jpa.show-sql: true
```

Run with `-Dspring.profiles.active=dev`. Profiles let one build behave differently per environment without rebuilding.

Real-time example — this is your SIT and UAT workflow. The same artefact is promoted through dev, SIT, UAT and production. Only the profile changes: different database URLs, different queue names, SQL logging on in dev and off in production. That is what makes it the *same build* being tested and released, rather than a rebuild per environment — which is the whole point of promoting a tested artefact.

```
@ConfigurationProperties(prefix = "app.payments")    // type-safe, grouped config
public class PaymentProperties { private int timeoutMs; private String vendorBaseUrl; }
```

Q8. Spring Boot Actuator

Adds production endpoints: `/actuator/health` (used by load balancers and Kubernetes to decide if the instance is alive), `/actuator/metrics`, `/actuator/info`.
Say that you would **secure these**, because they expose internals.

Say this.

"Actuator exposes operational endpoints — health, metrics and info. The health endpoint is the practical one: a load balancer or container orchestrator calls it to decide whether the instance should receive traffic, and it can include the database and message broker status, so an instance that has lost its database is taken out of rotation automatically.

I would secure these endpoints, because they expose internals. On a banking system I would not have them open."

Q9. How does auto-configuration actually work?

A step above the basics, and a good one to know:

"Spring Boot ships configuration classes annotated with conditions such as `@ConditionalOnClass` and `@ConditionalOnMissingBean`. At startup it evaluates those conditions against what is on the classpath and what you have already defined. So if H2 is on the classpath and I have not declared a `DataSource`, it creates one — but the moment I define my own bean, `@ConditionalOnMissingBean` backs off and mine wins. That is why defaults never fight your own configuration."

Q10. Testing a Spring Boot application

```
@WebMvcTest(TxnController.class)           // controller layer only, fast
class TxnControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean TxnService service;           // replaces the real bean

    @Test void returns404WhenMissing() throws Exception {
        when(service.findById(1L)).thenReturn(new TransactionNotFoundException(1L));
        mockMvc.perform(get("/api/transactions/1")).andExpect(status().isNotFound());
    }
}
```

- `@SpringBootTest` loads the whole context — integration tests, slower.
- `@WebMvcTest` / `@DataJpaTest` load one slice — much faster.
- `@MockBean` swaps a bean for a Mockito mock.

Your resume mentions unit tests for state-transition validation, so be ready to say what you actually tested and why.

Say this.

"I write mostly slice tests rather than loading the whole context, because they are much faster and a fast suite is one people actually run. `@WebMvcTest` for the controller layer with the service mocked, `@DataJpaTest` for repository queries against an in-memory database, and plain JUnit with Mockito for service logic, which needs no Spring at all. `@SpringBootTest` I keep for a small number of genuine end-to-end paths.

What I test first is the state transitions — that a rejected request cannot be approved, that an already-approved one cannot be approved twice. Those are the rules that cost money if they break."

✓ Check yourself before moving on

1. Explain IoC and DI, and why constructor injection is preferred — three reasons.
2. Explain what a singleton bean means for thread safety.
3. Write a `@RestControllerAdvice` handler from memory.
4. Explain how auto-configuration decides what to configure.

04 · JPA, Hibernate and the N+1 Problem

Time needed: 75 minutes

This is the single highest-value file for a backend interview at a bank. N+1 is the most common real performance bug in a Spring Boot and Hibernate application, and being able to explain it precisely — how it happens, how you would *detect* it, and four ways to fix it — is what separates someone who has used Hibernate from someone who has only configured it.

Your resume says you refactored code to improve performance and did root cause analysis.

Section Q3 has a template to fill with your own real example. Fill it in tonight with something that actually happened on Tradu. Do not borrow a number from anywhere.

| First, the context: what is JPA and what is Hibernate?

Your database stores **rows in tables**. Your Java code works with **objects**. Someone has to translate between them. Doing it by hand means endless JDBC boilerplate: open a connection, build SQL, loop the ResultSet, map each column to a field, close everything.

ORM (Object-Relational Mapping) automates that translation.

- **JPA** is the *specification* — a set of interfaces and annotations. It is a standard, not an implementation.
- **Hibernate** is the most common *implementation* of that specification.
- **Spring Data JPA** is a layer above both that generates repository implementations from method names, so you rarely write queries at all.

"JPA is the specification, Hibernate is the implementation, and Spring Data JPA is the abstraction on top that generates the repository code. Coding against JPA interfaces means I could switch the provider without rewriting my entities."

Q1. Entity mapping basics

```
@Entity
@Table(name = "transactions")
public class Transaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 20)
    private String accountNumber;

    private BigDecimal amount;

    @Enumerated(EnumType.STRING)    // store "PENDING", not 0 - ordinals break on reorder
    private TxnStatus status;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

⚠ `@Enumerated(EnumType.STRING)` is worth mentioning. The default is `ORDINAL`, which stores the enum's position as a number — so inserting a new value in the middle of the enum silently corrupts every existing row. In banking data that is a serious bug.

Say this.

"`@Entity` marks the class as mapped to a table and `@Id` is the primary key, with `@GeneratedValue` telling Hibernate the database generates it. `@Column` constrains the mapping, and `@ManyToOne` with `@JoinColumn` maps the foreign key.

Two details I always set explicitly. `@Enumerated(EnumType.STRING)`, because the default stores the enum's numeric position — so if someone inserts a new value in the middle of the enum, every existing row silently means something different. And `BigDecimal` for money, never `double`, because floating point cannot represent decimal fractions exactly and the rounding errors accumulate."

Q2. • Lazy vs eager loading

- **EAGER** — the related entity is loaded immediately, with the parent.
- **LAZY** — a proxy is put in place, and the real data is only fetched when you first access it.

The defaults, which they may ask you to recite:

Relationship	Default
@OneToOne	EAGER
@ManyToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

Say this:

"I set @ManyToOne to LAZY explicitly, because the default is EAGER and that means every time I load a transaction I also load its customer, even when I only wanted the amount. On a list of a thousand transactions that is a thousand unnecessary joins or queries."

Real-time example. Transaction has a @ManyToOne to Customer and to Account.

A transaction list screen shows only reference, amount and status — no user data at all. With the EAGER default, every single row still dragged its customer and account across, which is a large part of why that page was slow. Setting them LAZY meant the list query loaded only what the screen actually displays.

The catch with LAZY: if you access a lazy field after the persistence context is closed, you get LazyInitializationException. The correct fixes are to fetch what you need in the query (a join fetch), or map to a DTO inside the transaction. The wrong fix is spring.jpa.open-in-view=true, which keeps the session open for the whole request and hides the problem while making it worse.

Q3. ●● The N+1 problem — your story

This is the most important answer in this file. Be able to explain it in 60 seconds.

What it is

```
List<Transaction> txns = txnRepository.findAll();           // 1 query - gets 100 transactions

for (Transaction t : txns) {
    System.out.println(t.getCustomer().getName());         // 1 query EACH - 100 more
}
```

One query to get the list, then N more queries — one per row — to load each row's relation. Hence **N+1**. With 100 transactions that is **101 database round trips** instead of one or two.

Why it is so slow

Each round trip has network latency. Even at 5ms each, 100 extra queries is half a second of pure waiting, and it grows linearly with the data. This is why a report that

was fine in testing crawls in production — the row count went up.

How to find it

"Turn on `spring.jpa.show-sql` or set the Hibernate SQL logger to DEBUG, then load the page and count the queries. If you see the same query repeating with a different ID, that is N+1. I also looked at execution plans to confirm which queries were actually costly."

How to fix it — know all four

1. JOIN FETCH — the usual fix.

```
@Query("SELECT t FROM Transaction t JOIN FETCH t.customer WHERE t.status = :status")
List<Transaction> findByStatusWithCustomer(@Param("status") TxnStatus status);
```

One query, everything loaded.

2. `@EntityGraph` — the declarative version.

```
@EntityGraph(attributePaths = {"customer", "account"})
List<Transaction> findByStatus(TxnStatus status);
```

3. Batch fetching — turns N queries into N/batch queries.

```
spring.jpa.properties.hibernate.default_batch_fetch_size: 25
```

4. A DTO projection — the best option for a read-only report.

```
@Query("SELECT new com.bank.dto.TxnReportDto(t.id, t.amount, c.name) " +
        "FROM Transaction t JOIN t.customer c WHERE t.status = :status")
List<TxnReportDto> findReportData(@Param("status") TxnStatus status);
```

This selects only the columns you need, avoids loading full entities into the persistence context, and skips dirty-checking entirely.

⚠ **The trap with JOIN FETCH:** fetching **two** collections in one query produces a cartesian product. Fetch one collection at a time, or use batch fetching for the second. Mentioning this unprompted marks you out.

● Your version of the answer — fill this in tonight

Do **not** recite someone else's numbers. Use this template with something real from Tradu:

"[Where it showed up — an endpoint or screen that was slower than it should have been.] [How I found it — I turned on SQL logging / looked at the logs and saw the same query repeating once per row.] [What I changed — a join fetch, an `@EntityGraph`, a DTO projection, or setting the association to lazy.] [What the effect was — if you measured it, say the number; if you did not, say it plainly: 'it went from one query per row to a single query'.]"

If you have never personally fixed an N+1, say so and answer on the mechanism instead:

"I have not had to fix one myself yet, but I know what it is and how I would find it. If an endpoint were slow I would turn on SQL logging and count the queries. If I saw the same query repeating once per row, that is N+1 — the list loads in one query and then each row's relation loads separately. I would fix it with a join fetch or an `@EntityGraph`, or a DTO projection if it is a read-only screen, then re-check the query count to confirm."

That answer scores well. It is honest, it shows you understand the mechanism, and it describes a method rather than a memorised anecdote. An invented story collapses on the second follow-up; this one does not.

Things you genuinely can claim from your resume, and should:

- Refactoring existing code for maintainability, readability and performance.
- Adding logging to improve observability and make debugging faster.
- Root cause analysis on defects, within an Agile/JIRA workflow.
- JUnit 5 and Mockito tests to reduce regression risk.

Q4. • `@Transactional`

```
@Transactional
public void settleTransaction(Long id) {
    Transaction txn = repo.findById(id).orElseThrow();
    txn.setStatus(SETTLED);           // no save() needed - dirty checking
    auditService.record(txn);
    // commit on normal return, rollback on a RuntimeException
}
```

The three things they ask:

1. What does it actually do? Spring creates a **proxy** around the bean. The proxy opens a transaction before the method, commits after it returns, and rolls back on failure.

2. When does it roll back? By default on **unchecked** exceptions (`RuntimeException` and `Error`) only. A checked exception **commits** unless you say `@Transactional(rollbackFor = Exception.class)`. This surprises people and is a favourite question.

3. • Why does `@Transactional` not work when you call the method from inside the same class?

"Because it works through a proxy. An external caller goes through the proxy, so the transaction starts. But an internal call — `this.otherMethod()` — bypasses the proxy entirely and calls the real method directly, so no transaction is started. The fix is to move that method into a separate bean, so the call goes through the proxy again."

That is a senior-level answer and interviewers remember it.

Propagation, briefly: `REQUIRED` is the default — join the existing transaction or start one. `REQUIRES_NEW` suspends the current one and starts an independent one, which is what you want for audit logging that must persist even if the main work rolls back.

Real-time example — the clearest use of `REQUIRES_NEW`. A withdrawal fails validation halfway through and the main transaction rolls back, which is correct: no partial state is written. But the **audit record of the attempt must survive**, because compliance needs to know it was attempted. If the audit insert joins the main transaction it is rolled back with everything else. `REQUIRES_NEW` on the audit method gives it its own transaction that commits independently.

On a regulated financial platform that is not a nicety — an audit trail that disappears whenever something fails is worse than no audit trail, because you believe it.

Isolation levels, in one line each:

- `READ_UNCOMMITTED` — can see uncommitted data (dirty reads). Rarely used.
- `READ_COMMITTED` — only committed data. The usual default in PostgreSQL and Oracle.
- `REPEATABLE_READ` — the same row reads the same within a transaction. MySQL's default.
- `SERIALIZABLE` — full isolation, slowest.

Real-time example — why isolation matters here. Two reviewers open the same onboarding case and click Approve at the same moment. Both transactions read the status as PENDING, both see it as approvable, and both write APPROVED — so it is approved twice and two approval records exist. That is a **lost update**. The same shape appears in a double-click on a withdrawal: two debits from one intent.

The fixes are optimistic locking with an `@Version` column, so the second write fails and can be retried, or a `SELECT ... FOR UPDATE` pessimistic lock. Optimistic is usually the right choice, because genuine collisions are rare and it does not hold database locks.

Q5. Hibernate caching

- **First-level cache** — the persistence context. **Always on**, scoped to one transaction. Loading the same entity twice in one transaction hits the database once.
- **Second-level cache** — optional, shared across transactions, needs a provider like Ehcache or Hazelcast. Good for reference data that rarely changes, such as a list of instrument and currency reference data. Not for data that changes often.
- **Query cache** — caches query results. Must be used with the second-level cache and is easy to get wrong.

Real-time example. Branch codes, currency codes and transaction-status descriptions are read on nearly every request and change perhaps twice a year. That is textbook

second-level cache material. Transactions themselves are **never** cached that way — the data changes constantly and stale financial data is worse than a slow query.

Say this.

"The first-level cache is the persistence context. It is always on and scoped to a single transaction, so loading the same entity twice in one transaction only hits the database once. That one is free and you get it whether you think about it or not.

The second-level cache is optional and shared across transactions, and it needs a provider like Ehcache. It is worth it for reference data that is read constantly and changes rarely. I would not cache transactional data that way — stale financial data is worse than a slow query, and cache invalidation across instances is where the bugs live."

Q6. Spring Data JPA repositories

```
public interface TxnRepository extends JpaRepository<Transaction, Long> {

    // Spring generates the query FROM THE METHOD NAME
    List<Transaction> findByStatusAndAmountGreaterThan(TxnStatus status, BigDecimal amt);

    Optional<Transaction> findByReferenceNumber(String ref);

    // your own JPQL when the name would get silly
    @Query("SELECT t FROM Transaction t WHERE t.createdAt BETWEEN :from AND :to")
    List<Transaction> findInRange(@Param("from") LocalDate from, @Param("to") LocalDate to);

    // native SQL when you need database-specific features
    @Query(value = "SELECT * FROM transactions WHERE MATCH(notes) AGAINST(:q)", nativeQuery = true)
    List<Transaction> search(@Param("q") String q);
}
```

Hierarchy: `Repository` → `CrudRepository` → `PagingAndSortingRepository` → `JpaRepository`. Each adds methods; `JpaRepository` adds JPA-specific ones like `flush()` and `saveAll()`.

Pagination, which you should mention for any list endpoint:

```
Page<Transaction> page = repo.findByStatus(PENDING, PageRequest.of(0, 20,
    Sort.by("createdAt").descending()));
```

Say this.

"Spring Data generates the implementation from the method name, so `findByStatusAndAmountGreaterThan` needs no code at all. I use derived queries while the name stays readable, and switch to `@Query` with JPQL once the name would become unreadable — which is usually around three conditions. Native SQL only when I need something database-specific.

The one thing I always add on a list endpoint is `Pageable`. Returning an unbounded collection is fine in testing and falls over in production when the table grows."

Q7. `save()` vs `saveAndFlush()`, and entity states

Three states: **transient** (a new object, not known to Hibernate), **persistent** (managed by the persistence context — changes are tracked automatically), **detached** (was persistent, but the context has closed).

Dirty checking is the thing to mention: for a managed entity you do not need to call `save()`. At flush time Hibernate compares the entity with its loaded snapshot and issues an UPDATE only for the fields that changed.

`saveAndFlush()` pushes the SQL to the database immediately instead of waiting for the transaction to commit. Useful when you need a generated ID within the same transaction.

Say this.

"An entity is transient when it is a plain new object Hibernate does not know about, persistent once it is managed by the persistence context, and detached after the context closes. The important consequence is **dirty checking**: while an entity is managed I do not need to call `save()` at all. I change a field, and at flush time Hibernate compares it with the snapshot it loaded and issues an update for only the changed columns.

`saveAndFlush` pushes the SQL immediately instead of waiting for the commit, which I use when I need a generated ID within the same transaction."

Q8. `findOne` / `getReferenceById` vs `findById`

`findById` hits the database and returns `Optional`. `getReferenceById` returns a **lazy proxy** without querying — useful when you only need to set a foreign key relationship and never read the object's fields. Touching a field on a proxy for a row that does not exist throws `EntityNotFoundException`.

Say this.

"`findById` goes to the database and returns an `Optional`. `getReferenceById` returns a lazy proxy without querying at all, which is what I want when I only need to set a foreign key — for example attaching an existing customer to a new transaction. There is no point loading the whole customer row just to write its ID.

The catch is that if the row does not exist you do not find out until you touch a field on the proxy, and then it throws `EntityNotFoundException` — possibly somewhere far from where you called it."

✓ Check yourself before moving on

1. Explain the N+1 problem, how you detect it, and **four** ways to fix it.
2. Give your own N+1 answer in under 60 seconds — either a real example from Tradu, or the honest mechanism-only version. Practise whichever is true.
3. Explain why `@Transactional` does nothing on an internal method call.
4. State the default rollback behaviour and how to change it.
5. Give the fetch-type defaults for all four relationship annotations.

05 · Microservices

Time needed: 60 minutes

Microservices is named in the role — Java, Spring Boot and Microservices — and it is in your resume summary and skills list. **So you will be asked, and you need to be ready for the depth question**, because your project description is a Spring Boot backend rather than a distributed system you designed.

Q7 deals with that directly, and honestly. Read it before the interview. The short version: do not claim to have architected a microservices platform. Claim what is true, then demonstrate that you understand the model and its trade-offs. At 1 year 8 months nobody expects you to have designed one. They do expect you to know why they exist and what they cost.

If you are short of time, the highest-value parts are **Q1** (the patterns and what you give up), **Q2** (how services communicate), **Q3** (circuit breaker) and **Q7**.

| First, the context: monolith vs microservices

The monolith

One deployable application containing everything: users, transactions, reporting, notifications. One codebase, one build, one database.

It is not a bad thing. It is simple to develop, simple to deploy, and a single transaction can span the whole system. Most applications should start here.

Where it hurts as it grows:

- One small change means redeploying everything.
- One memory leak in reporting takes down payments too.
- You cannot scale only the part that is busy — you scale the whole thing.
- A large team constantly blocks itself in one codebase.

Microservices

Split it into independent services, each owning one business capability, each with its own database, communicating over the network.

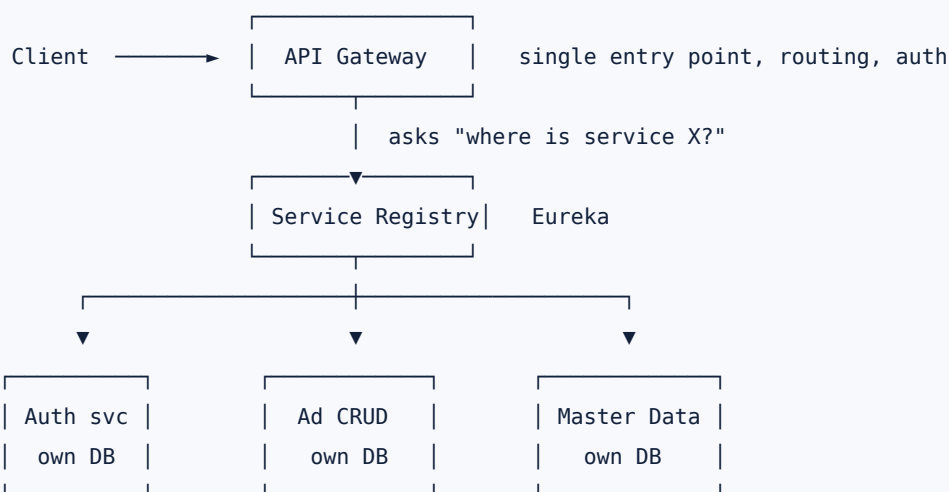
What you gain: independent deployment, independent scaling, fault isolation, and teams that can work without waiting for each other.

What you pay — say this part, because it shows judgement:

"You are trading code complexity for **operational and distributed-systems** complexity. Network calls fail, so you need retries and circuit breakers. There is no single transaction across services, so you need eventual consistency. Debugging spans several services, so you need distributed tracing and centralised logging. It is the right trade at a certain scale, but for a small application a well-structured monolith is usually the better engineering decision."

That last sentence is what separates a candidate who has *used* microservices from one who has *read* about them. Interviewers hear the buzzword version all day, and at a bank the costs are the half they care about — because they are the ones who live with them.

Q1. • The core patterns — and which you used



1. Service Registry / Discovery — Eureka.

Services are on dynamic hosts and ports, and instances come and go. Hard-coding addresses does not work. Each service **registers** itself with Eureka on startup and sends heartbeats; callers **look up** the current instances by service name.

"Without discovery I would be hard-coding hostnames, which breaks the moment a service scales or moves. With Eureka a caller asks for **ad-service** and gets a live list."

2. API Gateway.

One entry point for all clients. It handles **routing**, and centralises the things you do not want to repeat in every service: authentication, rate limiting, CORS, request logging.

"Without a gateway the client would need to know every service's address, and every service would need its own copy of the auth logic. The gateway puts the cross-cutting concerns in one place."

(This is exactly the "Microservices API Management" the JD asks for. Say the words.)

3. Database per service.

Each service owns its data; no other service touches its tables. This is what makes independent deployment real — you can change your schema without coordinating with other teams.

The cost: no joins across services, and no single ACID transaction. Which leads to the next two patterns.

4. Config Server — externalised configuration for all services, so a change does not require a rebuild.

5. Circuit breaker — see Q3.

| Q2. How do services talk to each other? ●

Synchronous — REST or gRPC. The caller waits for a reply.

```
@FeignClient(name = "master-data-service")    // declarative HTTP client
public interface MasterDataClient {
    @GetMapping("/categories/{id}")
    CategoryDto getCategory(@PathVariable Long id);
}
```

Simple, and you get an immediate answer. But it creates **temporal coupling**: if the callee is down, the caller fails too. Chains of synchronous calls multiply latency and failure probability.

Asynchronous — messaging over Kafka or RabbitMQ. The caller publishes an event and moves on.

"The publisher does not know or care who consumes it, and if a consumer is down the message waits in the queue. That removes the temporal coupling. The cost is eventual consistency — the effect is not immediate — and harder debugging, because the flow is no longer a straight line."

How to choose — the rule to state:

"If I need the answer to continue, synchronous. If I am telling the rest of the system that something happened, asynchronous. On a trading platform, calling a payment vendor to initiate a transfer is synchronous — I need the result. Telling downstream systems that a transaction settled, so they can log it for compliance and update reporting, is the second kind: those consumers should not be able to block the transaction from completing."

A good anchor for you: Tradu integrated multiple payment vendors. Vendor calls are the synchronous, must-have-an-answer case — and they are also exactly where timeouts, retries and circuit breakers matter, which leads into Q3.

Q3. • Resilience — circuit breaker, retry, timeout

The problem: service A calls B, B becomes slow. A's threads pile up waiting. A runs out of threads and dies, so C which calls A dies too. One slow service takes down the system. That is a **cascading failure**.

Circuit breaker — three states, know all three:

State	Behaviour
Closed	Normal. Calls pass through, failures are counted.
Open	Failure threshold exceeded. Calls fail immediately without trying.
Half-open	After a wait, let a few test calls through. Success closes it, failure opens it again.

```
@CircuitBreaker(name = "masterData", fallbackMethod = "defaultCategory")
public CategoryDto getCategory(Long id) { return client.getCategory(id); }

public CategoryDto defaultCategory(Long id, Throwable t) {
    return CategoryDto.unknown();    // degrade gracefully instead of failing
}
```

Resilience4j is the current library; Hystrix is the older one, now retired.

"Failing fast protects both sides. The caller does not exhaust its threads waiting, and the struggling service gets breathing room to recover instead of being hammered."

Real-time example. The advertisement service calls the master data service for category names. If master data goes down without a circuit breaker, every ad request hangs on that call until it times out, the ad service's thread pool fills, and the ad service stops serving requests it could have answered perfectly well. With a breaker and a fallback, ads still render — just with a placeholder category. **Degraded is much better than down**, and that sentence is the point of the whole pattern.

Also mention **timeouts** (never make a network call without one) and **retry with exponential backoff and jitter** — and that you only retry **idempotent** operations, because retrying a payment could charge someone twice.

Q4. • Distributed transactions and the Saga pattern

The problem: placing an order must reserve stock, take payment and create a shipment — in three different services with three databases. There is no `@Transactional` that spans them.

Saga — break it into a sequence of local transactions, each publishing an event that triggers the next. If a step fails, run **compensating transactions** to undo the earlier ones.

```
Order created → Payment taken → Stock reserved → Shipment booked
                                x fails
      ← refund payment ← cancel order      (compensation)
```

Two flavours: **choreography** (services react to each other's events — simple, but the overall flow is not written down anywhere) and **orchestration** (a coordinator drives the steps — clearer and easier to debug, but it is another component to run).

Mention **eventual consistency**: the system is briefly inconsistent, and the business has to accept that. In banking that decision belongs to the business, not the developer.

Real-time example. A withdrawal spanning services would be: debit the account, call the payment vendor, record the payout. If the vendor call fails after the account was already debited, you cannot roll back across two systems — you publish a compensating event that credits the account back and records why. The compensation is itself an auditable business action, not a silent undo. In a regulated financial system that is exactly how it has to work, because the reversal has to be explainable afterwards.

Idempotency matters here: a consumer may receive the same message twice, so processing must be safe to repeat. The usual approach is a unique message or request ID that you record and check before acting.

Say this.

"Once each service has its own database there is no distributed transaction to fall back on, so a business operation spanning services becomes a sequence of local transactions, each publishing an event that triggers the next. If a step fails, you cannot roll the earlier ones back, so you run **compensating transactions** that undo them as new business actions.

The two styles are choreography, where services react to each other's events, and orchestration, where a coordinator drives the sequence. Choreography is simpler to start but the overall flow is not written down anywhere, so I would prefer orchestration for anything with more than about three steps or anything auditable.

The thing to be explicit about with the business is that this is eventual consistency — there is a window where the system is partly updated. That is a business decision, not a technical one, and in finance it needs to be an informed one."

Q5. Observability — the question people forget

"Once a request crosses five services, a stack trace is not enough. You need three things. **Centralised logging**, so all logs land in one place — ELK or Splunk.

Distributed tracing with a correlation ID passed through every call, so you can follow one request end to end — Sleuth with Zipkin, or OpenTelemetry. And **metrics and health checks**, through Actuator and Prometheus, so you see latency and error rates per service."

Your resume mentions resolving production incidents by analysing logs, so this connects.

Q6. Other things they may ask

How do you version an API? URL versioning (`/api/v1/...`) is the most common and the most visible. The real rule is to make **additive** changes — adding an optional field is backwards compatible; removing or renaming one is not.

How do you handle security between services? The gateway validates the JWT once, then services trust the network boundary, or pass the token along and each validates it. Service-to-service can use mTLS.

What is the Strangler Fig pattern? How you migrate a monolith gradually: put a proxy in front, move one capability at a time to a new service, and route that path to the new one. The monolith shrinks until it disappears. Nobody sensible does a big-bang rewrite.

What is a bounded context? From domain-driven design — the boundary within which a model has one consistent meaning. It is the same way to decide **where to split services**: by business capability, not by technical layer.

"The wrong split is a 'database service' and a 'business logic service', because every change touches both. The right split is by business capability — orders, payments, notifications — so most changes stay inside one service."

How small should a service be? Not about lines of code:

"Small enough that one team can own it and understand it fully, and large enough that a typical change does not require modifying three services at once. If every feature spans multiple services, the boundaries are wrong."

Q7. ●● "Tell me about your microservices experience" — answer this honestly

This is the question to prepare most carefully, because your resume lists microservices but your project description is a Spring Boot backend for a trading platform. An interviewer will notice, and the worst outcome is claiming architectural ownership and then unravelling on the second follow-up.

The good news: at 1 year 8 months nobody expects you to have designed a distributed system. They expect you to be honest about your level and to understand the model. Honesty plus understanding beats a claim you cannot defend — every experienced interviewer has seen the second one, and they stop trusting everything else you said.

The answer to give

"I should be clear about the level I have worked at. On Tradu I built and consumed REST services within a Spring Boot backend — controllers, services, the business components behind them, and the APIs the React frontend consumed. The platform integrated multiple payment vendors, so I worked across service boundaries and with external APIs, including the failure handling that comes with them.

What I have not done is own the architecture of a multi-service system end to end — the service decomposition, the discovery and gateway layer. I understand how those work and why they are there, and I have been building toward that."

Then immediately show the understanding, because this is where you recover the ground:

"The way I think about it is that microservices buy independent deployment, independent scaling and fault isolation, and you pay for it with network failure, no distributed transaction, and much harder debugging. So for a small system a well-structured monolith is usually the better engineering decision, and the split makes sense when teams start blocking each other or one part needs to scale differently from the rest."

That paragraph is the whole answer. Most candidates recite the benefits. Saying the costs, unprompted, is what makes an interviewer think you have actually thought about it.

Follow-ups you should be ready for

- *"How would you split a monolith?"* → By **business capability**, not technical layer. Onboarding, payments, trading, reporting — not "the database service" and "the logic service", where every change touches both. Mention the **strangler fig** approach: proxy in front, move one capability at a time, never a big-bang rewrite.
- *"How do services find each other?"* → Service discovery, so addresses are not hard-coded and instances can scale and move. Eureka is the Spring Cloud implementation; in a Kubernetes environment the platform provides it.
- *"What is the API gateway for?"* → One entry point, and the place for cross-cutting concerns — authentication, rate limiting, routing — instead of duplicating them in every service.
- *"What happens when a downstream service is slow?"* → Timeouts always, then a circuit breaker and a fallback. See Q3. **Tie it to the payment vendors on your resume** — a vendor that hangs must not take your service down with it.

● The one thing not to do

Do not say "yes, I worked on microservices" and leave it there. If you are asked how many services, how they communicated, or how you handled a partial failure, and you have no answer, the interviewer discounts everything else you said. **Name your level first.** It costs you nothing and it buys credibility for the rest of the conversation.

✓ Check yourself before moving on

1. Explain monolith vs microservices **including what you give up**.
2. Explain the three circuit breaker states and why failing fast helps both sides.
3. Explain the Saga pattern and compensating transactions.
4. Explain why you would use messaging instead of a REST call, with your Kafka example.
5. Give the honest Q7 answer — your level, then the trade-offs — without notes.

06 · Kafka and Messaging

Time needed: 30 minutes — read for understanding, not for depth

⚠ **Calibrate this one honestly.** Your resume says "**Apache Kafka fundamentals**", which is the right word. Hold that line in the interview: know the model well, and say plainly that you have not run it in production. See file 11, section 5.2.

Why it is still worth 30 minutes: it is on your resume so it may be asked, and more importantly the *idea* behind it — moving slow work off the request thread — comes up in design questions whether or not Kafka is mentioned. Understanding why you would publish an event instead of making a synchronous call is a genuinely useful answer.

Read Q1 (Kafka vs RabbitMQ), Q3 (Kafka concepts) and Q4. Skip the rest if you are short of time.

First, the context: why messaging at all?

The problem you actually solved

An approval endpoint that sends an email synchronously:

```
@PostMapping("/approve/{id}")
public ResponseEntity<?> approve(@PathVariable Long id) {
    cardService.approve(id);           // 200 ms
    emailService.send(...);           // 2-4 seconds, and it can fail or hang
    return ResponseEntity.ok();       // the user has been waiting the whole time
}
```

Three problems:

1. **The user waits** for something they do not care about.
2. **If the mail server is slow, the API times out** — and to the user it looks like the approval failed, even though it succeeded.
3. **Bulk approvals multiply it.** 50 approvals means 50 sequential email sends inside the request.

The fix

```
@PostMapping("/approve/{id}")
public ResponseEntity<?> approve(@PathVariable Long id) {
    cardService.approve(id);           // 200 ms
    rabbitTemplate.convertAndSend("email.queue", new EmailMessage(...)); // ~1 ms
    return ResponseEntity.ok();       // returns immediately
}
```

A separate consumer picks the message up and sends the email. The API responds in milliseconds. If the mail server is down, the message waits in the queue and is delivered when it recovers, instead of being lost.

Say it this way:

"The email is not part of what the user is waiting for, so it should not be inside the request. Moving it onto a queue means the API returns immediately, and a mail server outage stops being an API failure — the messages wait in the queue until it recovers."

On a trading platform the same shape applies to confirmation emails, audit events and downstream reporting after a transaction settles. The rule is: **if the user does not need it to continue, it does not belong in the request.**

Q1. ●● Kafka vs RabbitMQ — when do you use which?

This is the question your resume invites. You used both, so you will be asked to justify it.

	RabbitMQ	Kafka
Model	Message broker / queue	Distributed event log
After consumption	Message is removed	Message stays for its retention period
Consumers	Usually one consumer per message	Many independent consumers read the same events
Replay	No	Yes — reset the offset and re-read
Routing	Rich — exchanges, routing keys, topics	Simple — topics and partitions
Ordering	Per queue	Per partition
Throughput	High	Very high — designed for streams
Best for	Task queues, work distribution, RPC-style	Event streaming, audit trails, multiple subscribers

The one-line distinction to memorise:

"RabbitMQ is a **postbox** — a message is delivered to a worker and then it is gone. Kafka is a **ledger** — events are appended and stay there, and any number of consumers can read them at their own pace, including re-reading from the past."

● Your answer, using your own project

This is the strongest version, because it is true and it shows you chose deliberately:

"I used them for two different jobs.

RabbitMQ for the email notifications. That is a task queue: one message, one worker, do the job, and the message is done. I did not need to keep it or replay it — I needed the work off the request thread so bulk approvals stopped timing out.

Kafka for publishing transaction state changes. That is an event stream, and several downstream consumers needed it — compliance logging and report generation, and potentially more later. With Kafka each consumer group reads independently at its own pace, and because the events are retained, a consumer that was down can catch up, and compliance can replay history if it needs to rebuild. With a queue, the first consumer would take the message and the others would never see it."

That last sentence is the technical heart of it. It shows you understand the difference rather than having used whatever the team had.

Q2. RabbitMQ concepts

Producer → Exchange → (routing rules) → Queue → Consumer

The **exchange** is the piece people forget. A producer never sends to a queue directly; it sends to an exchange, which decides which queues get a copy.

Exchange type	Routes by
Direct	Exact routing key match
Topic	Wildcard pattern — <code>txn.*.approved</code>
Fanout	Copies to every bound queue, ignoring the key
Headers	Header values

```
@RabbitListener(queues = "email.queue")
public void handle(EmailMessage msg) {
    emailSender.send(msg);           // ack on normal return; requeue or DLQ on exception
}
```

Acknowledgements. The broker only removes a message once the consumer acknowledges it. If the consumer crashes mid-processing, the message is redelivered — so processing must be **idempotent**.

Dead Letter Queue (DLQ). Where messages go after repeated failures, instead of being retried forever or dropped silently. Mentioning DLQs unprompted signals production experience:

"Without a DLQ a poison message either blocks the queue or gets retried indefinitely. With one, it moves aside after N attempts, gets alerted on, and someone can inspect it without holding up everything behind it."

Q3. Kafka concepts ●

Topic: transaction-events

└─ Partition 0 → [msg][msg][msg] ← ordered within the partition
└─ Partition 1 → [msg][msg]
└─ Partition 2 → [msg][msg][msg]

Topic — a named stream of events.

Partition — a topic is split into partitions, which is how Kafka scales. Order is guaranteed **within** a partition, not across the topic.

Offset — each consumer's position in a partition. Consumers commit their offsets, so they can resume where they left off.

Consumer group — consumers in the same group **share** the partitions, one partition per consumer at a time. Consumers in **different** groups each get **all** the messages.

That last point is the mechanism behind your answer in Q1: compliance and reporting are two different consumer groups, so both see every event.

● How do you guarantee ordering in Kafka?

"By partition key. All messages with the same key go to the same partition, and order is guaranteed within a partition. So for transaction events I would key by account or transaction ID, which guarantees that everything about one transaction is processed in order, while different transactions still spread across partitions for throughput."

Delivery guarantees — know the three:

- **At most once** — commit the offset before processing. Fast, may lose messages.
- **At least once** — commit after processing. **The common default.** May duplicate, so consumers must be idempotent.
- **Exactly once** — possible with Kafka transactions, but costly and complex.

"In practice most systems run at-least-once and make consumers idempotent, because exactly-once across a distributed system is expensive. For financial events I would record a processed-message ID and check it before acting, so a redelivery is a no-op."

Replication. Each partition has a leader and followers on other brokers. If the leader dies, a follower is promoted. That is Kafka's durability story.

Q4. Quick comparison questions

"Why not just use a database table as a queue?"

"It works at small scale, and people do it. But you end up building polling, locking, retries, dead-lettering and back-pressure yourself, and polling a table is wasteful and adds latency. A broker gives you all of that, plus push delivery."

"What happens if a message fails?"

Retry with backoff, then a dead letter queue after N attempts, then alert. Never silently swallow.

"What if the same message arrives twice?"

Idempotency: a unique message ID recorded on the consumer side, or design the operation so applying it twice has the same effect as once.

"How do you not lose messages?"

Producer side: acknowledgements and retries. Broker side: persistence and replication. Consumer side: acknowledge only **after** successful processing.

✓ Check yourself before moving on

1. Kafka vs RabbitMQ in one sentence each, then **your** reason for using both.
2. Explain consumer groups and why two groups both receive every event.
3. Explain how ordering works in Kafka and how you would key transaction events.
4. Explain why at-least-once delivery means consumers must be idempotent.
5. Tell the email-timeout story with the reasoning, in 45 seconds.

07 · Spring Security and JWT

Time needed: 45 minutes

Your resume lists Spring Security with role-based access control, and JWT in your microservices project. Expect a few questions, especially in a banking context.

| First, the context: authentication vs authorisation

- **Authentication** — *who are you?* Checking credentials.
- **Authorisation** — *what are you allowed to do?* Checking permissions.

You authenticate once; you authorise on every request. On a trading platform, authentication proves who the user is; authorisation decides whether they are a **trader** acting on their own account, a **compliance reviewer** who can approve or reject an onboarding case, or an **admin**.

| Q1. How does Spring Security work?

It sits in front of your application as a **chain of servlet filters**. Every request passes through the chain before reaching a controller.

```
Request → [ filters: authenticate, authorise, CSRF, CORS ] → DispatcherServlet → Controller
```

The key filter for a token-based API is one that reads the token, validates it, and places an `Authentication` object into the `SecurityContext`. Everything downstream then knows who the caller is.

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain chain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless API - see Q5
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/cards/approve/**").hasRole("APPROVER")
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

Say this.

"Spring Security is a chain of servlet filters that runs before the request reaches a controller. For a token-based API the key filter reads the `Authorization` header, validates the token, and puts an `Authentication` object into the `SecurityContext` — after which everything downstream knows who the caller is and what roles they hold.

The important point is that it runs **before** my code, so an unauthenticated request never reaches business logic at all. I do not have to remember to check in each method."

Q2. • Role-based access control — your story

Two ways to enforce it. Know both.

At the URL level, as above with `hasRole(...)`.

At the method level, which is more precise:

```

@PreAuthorize("hasRole('APPROVER')")
public CardDto approve(Long id) { ... }

@PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
public UserDto getUser(Long userId) { ... } // admins, or the user themselves

```

Requires `@EnableMethodSecurity`.

Your version:

"On Tradu the backend served a React frontend, and the rule that mattered was that a user must never be able to reach another user's account or perform an action their role does not permit. I worked on enforcing that at the API layer with Spring Security rather than relying on the UI, because a UI check only hides the button — anyone can call the endpoint directly with Postman. Authorisation has to be server side."

That last sentence is the point. In a banking interview it lands well.

⚠ **hasRole** vs **hasAuthority** : `hasRole("ADMIN")` looks for the authority `ROLE_ADMIN` — Spring adds the prefix for you. `hasAuthority("ROLE_ADMIN")` is the literal form. Mismatched prefixes are a very common bug.

Q3. • JWT — how it works

A JWT is three base64 sections separated by dots:

```
header.payload.signature

header    { "alg": "HS256", "typ": "JWT" }
payload   { "sub": "neelima", "roles": ["APPROVER"], "exp": 1735689600 }
signature HMACSHA256(base64(header) + "." + base64(payload), secret)
```

The flow

1. POST /login with credentials
2. Server verifies, signs a JWT, **returns** it
3. Client sends it **on** every request: Authorization: Bearer <token>
4. Server verifies the SIGNATURE and expiry - no **database** lookup needed
5. Request proceeds with the user's identity and roles

• Why use a JWT instead of a session?

"A session means the server stores state and every request looks it up, which is fine for one server but awkward when you scale — you need sticky sessions or a shared store like Redis. A JWT is **self-contained**: the user and roles are inside it, and the signature proves it has not been tampered with. So any instance of any service can validate it independently, with no shared session store. In a microservices setup that is the deciding factor."

• The two things people get wrong

1. A JWT is signed, not encrypted. Anyone can base64-decode the payload and read it. The signature only proves it has not been *altered*. **Never put anything sensitive in the payload** — no passwords, no account numbers.

2. You cannot easily revoke one. It is valid until it expires, so if a token is stolen it works until then. The usual answers: keep access tokens **short-lived** (15 minutes) with a longer-lived **refresh token**, and keep a small **blocklist** of revoked token IDs for logout. Say the trade-off rather than pretending it is solved.

Q4. Where should the token be stored on the client?

"An `httpOnly` cookie is safer than `localStorage`, because JavaScript cannot read an `httpOnly` cookie, so an injected script cannot steal it. Anything in `localStorage` is readable by any script on the page. The trade-off is that cookies need CSRF protection, whereas an `Authorization` header does not."

Q5. CSRF, and why it is disabled for APIs

CSRF is when another site makes the user's browser send a request to yours, relying on the browser automatically attaching the session cookie.

"It is disabled for a stateless API because the attack depends on the browser attaching credentials automatically. If the client sends an explicit `Authorization` header, another site cannot make the browser add it. If I were using cookie-based auth, CSRF protection would need to stay on."

Do not just say "we disable CSRF" — say *why*, or it looks like copied configuration.

Q6. Password storage

```
@Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

Never store plain text, and never a plain hash like MD5 or SHA-256. BCrypt is deliberately **slow** and **salts** each password automatically, which defeats rainbow tables and makes brute force expensive.

Say this.

"Passwords are stored as a BCrypt hash, never plain text and never a plain SHA or MD5 hash. Two properties matter. BCrypt salts each password automatically, so two users with the same password get different hashes and a precomputed rainbow table is useless. And it is deliberately **slow** with a tunable work factor, so brute forcing is expensive — which is the opposite of what you want from a general-purpose hash. That slowness is the feature."

Q7. Other things worth a sentence

- **OAuth2 vs JWT** — not alternatives. OAuth2 is the authorisation *framework*; a JWT is a token *format* often used within it.

- **CORS vs CSRF** — CORS is a browser rule about which origins may read a response. CSRF is an attack. They get confused constantly.
- **Method security vs URL security** — URL rules are coarse and can be bypassed if another entry point reaches the same service method. Method-level annotations protect the method itself.

Say this if they probe the distinctions.

"OAuth2 and JWT are not alternatives — OAuth2 is an authorisation framework describing how a token is obtained, and JWT is a token format often used inside it. CORS and CSRF also get confused: CORS is a browser rule about which origins may read a response, and CSRF is an attack where another site makes the browser send an authenticated request. And URL-level security is coarse — if another entry point reaches the same service method it is unprotected, so for anything sensitive I put `@PreAuthorize` on the method itself."

✓ Check yourself before moving on

1. Explain the JWT structure and the full request flow.
2. Explain why a JWT suits microservices better than server-side sessions.
3. Explain why a JWT is not secret, and what that means for what you put in it.
4. Explain why CSRF is disabled for a stateless API.
5. Tell your RBAC story, ending on why authorisation must be server side.

08 · SQL and Query Optimisation

Time needed: 75 minutes

Your resume claims SQL tuning, indexing and execution plan analysis. That is unusual at this level and it is genuinely valuable — but it also means you **will** be tested on it. An interviewer who sees "identified and removed N+1 queries, added strategic indexing" will ask you to explain indexing properly.

Q1. Joins — know all four

```
-- INNER: only rows that match in both tables
SELECT t.id, c.name FROM transactions t
INNER JOIN customers c ON t.customer_id = c.id;

-- LEFT: every row from the left, NULLs where the right has no match
SELECT c.name, t.id FROM customers c
LEFT JOIN transactions t ON t.customer_id = c.id;    -- includes customers with no txns

-- RIGHT: the mirror image. Rarely used - people rewrite it as a LEFT JOIN.
-- FULL OUTER: everything from both sides. MySQL does not support it directly.
```

The classic question: "Find customers with no transactions."

```
SELECT c.* FROM customers c
LEFT JOIN transactions t ON t.customer_id = c.id
WHERE t.id IS NULL;
```

The `WHERE ... IS NULL` after a LEFT JOIN is the pattern. Learn it.

⚠ **A trap worth knowing:** putting a condition on the right-hand table in the `WHERE` clause of a LEFT JOIN silently turns it into an INNER JOIN, because `NULL` fails the condition. If you want to filter the joined table without losing left rows, the condition belongs in the `ON` clause.

Say this.

"An inner join returns only rows that match on both sides. A left join returns every row from the left table and nulls where the right has no match, which is what I use when the absence is the thing I care about — customers with no transactions, or requests with no approval yet. Right joins are the mirror image and people usually rewrite them as left joins for readability.

The trap is putting a condition on the right-hand table in the `WHERE` clause of a left join. Null fails the condition, so those rows are dropped and you have silently turned it back into an inner join. The condition has to go in the `ON` clause."

Q2. ●● Indexing — the answer that matters most here

What an index actually is

"An index is a separate sorted structure, usually a **B-tree**, that maps column values to row locations. Without one, the database does a **full table scan** — it reads every row to find matches, which is $O(n)$. With one, it walks the tree, which is $O(\log n)$. It is the same reason you use the index at the back of a book instead of reading every page."

When an index helps

- Columns in a `WHERE` clause
- Columns you `JOIN` on — **especially foreign keys, which are not indexed automatically in MySQL's InnoDB unless you add them**
- Columns in `ORDER BY` and `GROUP BY`

● The cost — always mention this, it is what shows real understanding

"Indexes are not free. Every `INSERT`, `UPDATE` and `DELETE` has to update every index on that table, so over-indexing slows down writes. They also take disk space. On a write-heavy table I index deliberately, not by default — the right number is the smallest set that serves the actual query patterns."

Saying only "indexes make queries faster" is the answer of someone who read it. Saying "and they slow down writes, so I add them deliberately" is the answer of someone who has tuned a real system.

Composite indexes and the left-most prefix rule

```
CREATE INDEX idx_txn_status_date ON transactions (status, created_at);
```

This index serves:

- `WHERE status = 'PENDING'` ✓
- `WHERE status = 'PENDING' AND created_at > '2026-01-01'` ✓

But **not** `WHERE created_at > '2026-01-01'` alone ✗ — because the index is sorted by `status` first. This is the **left-most prefix rule**, and it is a favourite question. Column order in a composite index matters.

Real-time example. A transaction history query filters by status and then a date range: `WHERE status = 'PENDING' AND created_at BETWEEN ? AND ?`. The index should be `(status, created_at)` in that order — the equality column first, the range column second. Reversed, as `(created_at, status)`, it is far less effective, because the range scan happens first and status cannot narrow it.

What stops an index being used

```
WHERE YEAR(created_at) = 2026      -- ✗ a function on the column defeats the index
WHERE created_at >= '2026-01-01'  -- ✓ same meaning, index usable

WHERE account_number LIKE '%1234'  -- ✗ a leading wildcard cannot use a B-tree
WHERE account_number LIKE '1234%'  -- ✓ prefix match is fine
```

Mentioning these two unprompted is a strong signal.

Clustered vs non-clustered

- **Clustered** — the table rows are physically stored in this order. One per table; in InnoDB it is the primary key.
- **Non-clustered** — a separate structure pointing back to the row.

Covering index

An index containing every column a query needs, so the database answers it from the index alone and never touches the table. Sometimes the biggest single win available.

Real-time example. A dashboard count of pending transactions per currency needs only `status` and `currency`. An index on `(status, currency)` covers it, so the database answers from the index and never reads a table row. On a large transactions table that is the difference between milliseconds and seconds.

Q3. • EXPLAIN — how you actually diagnose

Your resume says you analysed execution plans. Be ready to describe the process.

```
EXPLAIN SELECT t.id, c.name FROM transactions t
JOIN customers c ON t.customer_id = c.id
WHERE t.status = 'PENDING';
```

What you look for:

Column	What matters
type	<code>ALL</code> = full table scan, the thing to fix. <code>ref</code> or <code>range</code> is good. <code>const</code> / <code>eq_ref</code> is best.
key	Which index was actually used. <code>NULL</code> means none.
rows	Estimated rows examined. A big number here is the problem.
Extra	<code>Using filesort</code> or <code>Using temporary</code> mean extra work. <code>Using index</code> is a covering index — good.

Say the process, not just the tool:

"I start from the slow query log or the application timings to find *which* query is slow — there is no point optimising the wrong one. Then I run `EXPLAIN` on it. If the type is `ALL` and the row count is large, it is doing a full scan, and I look at what the `WHERE` and `JOIN` columns are and whether an index exists. After adding one I re-run `EXPLAIN` to confirm it is actually being used, and re-measure the query time. Adding an index without checking the plan afterwards is guessing."

Q4. Normalisation, and when to break it

- **1NF** — atomic values, no repeating groups.
- **2NF** — 1NF, and no partial dependency on part of a composite key.
- **3NF** — 2NF, and no non-key column depending on another non-key column.

"Normalisation removes duplication, so there is one place to update a fact and no inconsistency. The cost is more joins. **Denormalisation** deliberately duplicates data to avoid expensive joins on read-heavy paths — for example storing a customer name on a report table so a reporting query does not join. It is a trade: faster reads, but now you have two copies to keep in sync."

Q5. Common query questions

Second-highest salary — asked constantly:

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

-- or, more generally, the Nth highest
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Find duplicates:

```
SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1;
```

● **WHERE vs HAVING** : `WHERE` filters **rows before** grouping. `HAVING` filters **groups after** aggregation. You cannot use an aggregate in `WHERE`.

UNION vs UNION ALL : `UNION` removes duplicates, which costs a sort. `UNION ALL` keeps everything and is faster. Use `UNION ALL` unless you actually need deduplication.

DELETE vs TRUNCATE vs DROP : `DELETE` removes rows, can have a `WHERE`, is logged and can be rolled back. `TRUNCATE` empties the table quickly, no `WHERE`, resets auto-increment. `DROP` removes the table itself.

Window functions, worth knowing one:

```
SELECT name, salary, department,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept  
FROM employees;
```

Unlike `GROUP BY`, a window function keeps every row and adds a computed column.

Say this when they give you a query to write.

"Let me state what I am aiming for first — I want the second highest distinct salary, so I need to handle duplicates, and decide what should happen if there is no second value."

Then write it. The pattern that scores is: restate, mention the edge case, write, then say the complexity or the index it would need. For the duplicates query, mention that `GROUP BY ... HAVING COUNT(*) > 1` is the standard shape and that on a large table the grouped column wants an index.

Q6. ACID

- **Atomicity** — all of the transaction, or none of it.
- **Consistency** — the database moves from one valid state to another; constraints hold.
- **Isolation** — concurrent transactions do not corrupt each other.
- **Durability** — once committed, it survives a crash.

The banking example makes this concrete: a transfer is a debit and a credit. Without atomicity, a crash between them destroys money. Use that example — it fits your domain.

Say this — use the banking example, it is your domain.

"A transfer is the clearest example. Debiting one account and crediting another must be **atomic** — both or neither, because a crash between them destroys money.

Consistency means constraints still hold afterwards, so the ledger still balances.

Isolation means two concurrent transfers on the same account cannot interleave and produce a wrong balance. And **durability** means once it is committed it survives a crash — the customer has been told it happened, so it has to have happened.

That is also why I would not move something like a ledger entry to eventual consistency without the business explicitly agreeing to it."

Q7. SQL vs NoSQL — you have MongoDB on your resume

"I would use a relational database when the data is structured and relational and I need ACID transactions — which is most of banking. MongoDB suits flexible or evolving schemas, document-shaped data, and cases where horizontal scaling matters more than joins. The real question is the access pattern: if I am constantly joining, that is a relational shape, and forcing it into documents means duplicating data everywhere."

Q8. Pagination at the database level

```
SELECT * FROM transactions ORDER BY created_at DESC LIMIT 20 OFFSET 0;
```

"OFFSET pagination degrades on deep pages, because the database still has to scan and discard everything before the offset. Page 5,000 is slow. **Keyset pagination** — `WHERE created_at < :lastSeen ORDER BY created_at DESC LIMIT 20` — stays fast because the index seeks straight to the position."

That is a strong detail for a reporting-heavy system, which yours was.

✓ Check yourself before moving on

1. Explain what an index is, **and its cost on writes**.
2. Explain the left-most prefix rule with the composite index example.
3. Give two things that stop an index being used.
4. Describe your EXPLAIN process end to end.
5. Write the "customers with no transactions" query from memory.
6. Explain `WHERE` vs `HAVING`.

09 · Coding Round

Time needed: 90 minutes. Write the code, do not read it.

Reading solutions feels productive and is nearly worthless. Open an IDE and type these from an empty file.

| What to expect

The coding bar for a backend role at this level is **moderate**. They are not asking for dynamic programming. Expect:

- String and array manipulation
- Collections and HashMap usage
- Java 8 stream problems
- One "write a small class" or design question
- Sometimes SQL instead of code

They are checking: can you write correct, compiling Java; do you handle edge cases; do you talk while you work. Not whether you know a clever trick.

The process, which is half the marks:

1. **Restate the problem.** "So I need to return the first character that appears once, and null if there isn't one?"
 2. **Ask about edge cases** *before* coding. Empty input? Nulls? Case sensitivity?
 3. **Say your approach out loud.** "I'll count with a LinkedHashMap so I keep order, then find the first with a count of one."
 4. **Then write it.**
 5. **Trace one example** out loud when done.
 6. **State the complexity.** They almost always ask, so beat them to it.
-

| Problem 1 ● — First non-repeating character

The most common Java string question in Indian interviews.

```

public static Character firstNonRepeating(String input) {
    if (input == null || input.isEmpty()) return null;

    Map<Character, Integer> counts = new LinkedHashMap<>(); // preserves insertion order
    for (char c : input.toCharArray()) {
        counts.merge(c, 1, Integer::sum);
    }

    for (Map.Entry<Character, Integer> e : counts.entrySet()) {
        if (e.getValue() == 1) return e.getKey();
    }

    return null;
}

```

Say: "LinkedHashMap, not HashMap, because I need insertion order to find the *first* one. Two passes, so $O(n)$ time and $O(k)$ space where k is the distinct character count."

Problem 2 ● — Reverse a string, and check a palindrome

```

public static String reverse(String s) {
    if (s == null) return null;
    return new StringBuilder(s).reverse().toString();
}

// they often want it without the library method
public static String reverseManual(String s) {
    char[] chars = s.toCharArray();
    int left = 0, right = chars.length - 1;
    while (left < right) {
        char tmp = chars[left];
        chars[left++] = chars[right];
        chars[right--] = tmp;
    }
    return new String(chars);
}

public static boolean isPalindrome(String s) {
    String clean = s.replaceAll("[^A-Za-z0-9]", "").toLowerCase();
    int left = 0, right = clean.length() - 1;
    while (left < right) {
        if (clean.charAt(left++) != clean.charAt(right--)) return false;
    }
    return true;
}

```

The two-pointer palindrome is better than reversing and comparing, because it can exit early and uses no extra string. Say that.

Problem 3 ● — Duplicates in an array

```
// find duplicates
public static Set<Integer> findDuplicates(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> dupes = new HashSet<>();
    for (int n : nums) {
        if (!seen.add(n)) dupes.add(n); // add() returns false if already present
    }
    return dupes;
}

// remove duplicates, keeping order
public static List<Integer> removeDuplicates(List<Integer> nums) {
    return new ArrayList<>(new LinkedHashSet<>(nums));
}
```

`seen.add(n)` returning false is the neat part — one lookup instead of `contains` then `add`.

Problem 4 ● — Stream problems

These are very likely, because your resume says Java 8.

```

// count word frequency
Map<String, Long> freq = Arrays.stream(sentence.toLowerCase().split("\\s+"))
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));

// second highest number
Optional<Integer> second = nums.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();

// group employees by department
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// highest paid employee per department
Map<String, Optional<Employee>> topPerDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.maxBy(Comparator.comparing(Employee::getSalary))));

// sum of even numbers
int sum = nums.stream().filter(n -> n % 2 == 0).mapToInt(Integer::intValue).sum();

// sort a map by value, descending
Map<String, Integer> sorted = map.entrySet().stream()
    .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
        (a, b) -> a, LinkedHashMap::new));

```

That last one is worth practising — the four-argument `toMap` with `LinkedHashMap::new` is needed to keep the sort order, and people always forget it.

Problem 5 — Two sum

```

public static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();    // value -> index
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) return new int[]{ seen.get(need), i };
        seen.put(nums[i], i);
    }
    return new int[]{};
}

```

Say: "The brute force is two nested loops, $O(n^2)$. Storing what I have seen in a HashMap makes the lookup $O(1)$, so it becomes one pass, $O(n)$ time and $O(n)$ space. That is the classic time-for-space trade."

Talking about the brute force *first*, then improving it, is exactly what they want.

Problem 6 — Fibonacci and factorial

```
// iterative - preferred, no stack growth
public static long fib(int n) {
    if (n <= 1) return n;
    long prev = 0, curr = 1;
    for (int i = 2; i <= n; i++) {
        long next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}

// recursive with memoisation, if they ask for recursion
public static long fibMemo(int n, Map<Integer, Long> memo) {
    if (n <= 1) return n;
    return memo.computeIfAbsent(n, k -> fibMemo(k - 1, memo) + fibMemo(k - 2, memo));
}
```

Naive recursion is $O(2^n)$ and will be called out. Memoisation makes it $O(n)$.

Problem 7 — Reverse words, count vowels, anagram

```
public static String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    Collections.reverse(Arrays.asList(words));
    return String.join(" ", words);
}

public static boolean isAnagram(String a, String b) {
    if (a == null || b == null || a.length() != b.length()) return false;
    char[] x = a.toCharArray(), y = b.toCharArray();
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

For anagram, mention the alternative: counting characters in a map is $O(n)$ versus $O(n \log n)$ for sorting. Naming the trade-off scores.

Problem 8 — "Write a class" questions

Singleton, thread safe:

```

public class ConfigManager {
    private static volatile ConfigManager instance;
    private ConfigManager() { }

    public static ConfigManager getInstance() {
        if (instance == null) { // first check, no lock
            synchronized (ConfigManager.class) {
                if (instance == null) { // second check, with lock
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}

```

This is **double-checked locking**. `volatile` is essential — without it another thread can see a partially constructed object due to instruction reordering.

Say the better answer too: "In a Spring application I would not write this at all. A `@Component` is already a singleton in the container, and that is easier to test than a static instance. The enum singleton is also simpler if I truly need one outside Spring."

Immutable class:

```

public final class Money { // final: cannot be subclassed
    private final BigDecimal amount; // final fields
    private final String currency;

    public Money(BigDecimal amount, String currency) {
        this.amount = amount;
        this.currency = currency;
    }
    public BigDecimal getAmount() { return amount; } // no setters
}

```

The four rules: make the class `final`, all fields `private final`, no setters, and if a field is mutable (a `Date`, a `List`) return a **defensive copy** from the getter rather than the field itself.

Java-specific traps they enjoy

"What does this print?"

```
Integer a = 127, b = 127;
Integer c = 128, d = 128;
System.out.println(a == b); // true - Integer cache covers -128..127
System.out.println(c == d); // false - outside the cache, different objects
System.out.println(c.equals(d)); // true
```

```
String s1 = "abc";
String s2 = "abc";
String s3 = new String("abc");
s1 == s2 // true - string pool
s1 == s3 // false
s1.equals(s3) // true
s1 == s3.intern() // true - intern() returns the pooled instance
```

```
System.out.println(0.1 + 0.2 == 0.3); // false - floating point
// which is exactly why money uses BigDecimal
```

✓ Practice plan for this file

Type problems **1, 3, 4 and 5** from an empty file, no copy-paste. If you can write those four while explaining what you are doing, you can handle this round.

10 · Backend System Design

Time needed: 75 minutes

At under two years nobody will ask you to design Netflix. But expect one design question, possibly framed as "how would you build X" or "how would you improve this", and expect them to be watching **how you think** rather than whether you produce a perfect architecture.

At a bank, the things that earn marks are the ones most candidates skip: what happens when a dependency fails, how you would know in production, and how it gets released safely.

The framework — use it for every design question

- | | |
|----------------------|---|
| 1. CLARIFY | Ask 2-3 questions. Never start designing immediately. |
| 2. SCOPE | State the core use cases you will design for . |
| 3. API | What endpoints , what request and response shapes. |
| 4. DATA MODEL | Tables or documents, and the key relationships. |
| 5. FLOW | Walk one request end to end . |
| 6. SCALE | What breaks first, and what you would do about it. |

Step 1 is the one candidates skip, and it is the one the JD explicitly asks for.

"Problem definition" is literally in the job description.

Clarifying questions that fit almost anything:

- "Roughly what volume are we designing for — hundreds a day or millions?"
- "Is this read-heavy or write-heavy?"
- "Does this need to be strongly consistent, or is eventual consistency acceptable?"
- "Is there an existing system I am integrating with, or is this greenfield?"

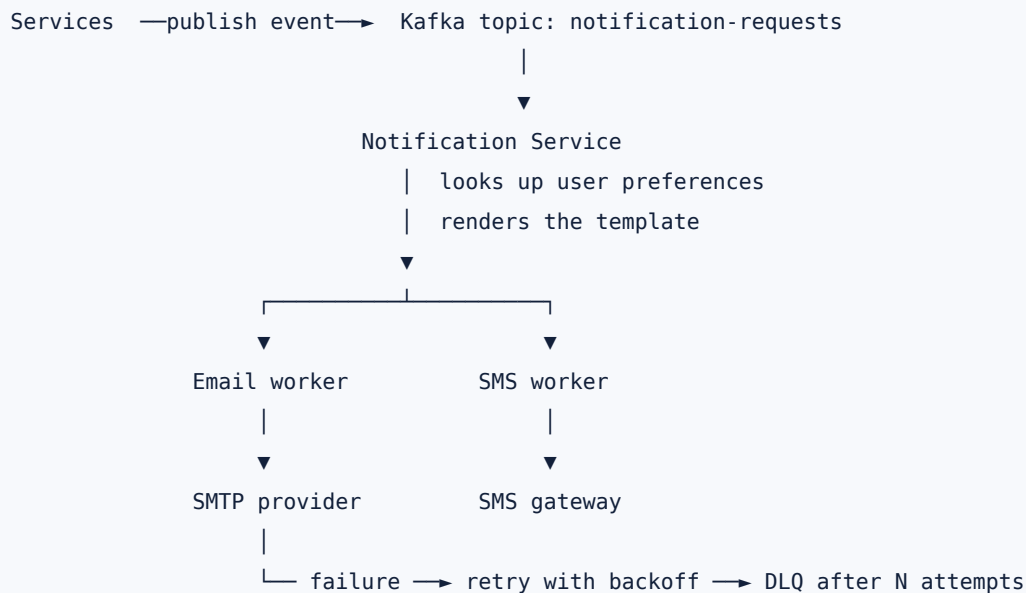
That third question is a strong one to ask in a banking context, because the answer is usually "strongly consistent for money, eventually consistent for notifications" — and saying that shows domain judgement.

Design 1 • — A notification service

Most likely for you, because your resume already describes half of it.

Clarify: Which channels — email, SMS, push? Do we need delivery status? Are there priorities, or is everything equal? Roughly what volume?

The design:



The points to make:

- **Asynchronous, always.** The calling service must never wait for an email. This is exactly what you did with RabbitMQ.
- **Preferences and templates** live in the notification service, not scattered across callers. One place to change how something is worded.
- **Retry with exponential backoff**, then a dead letter queue, then an alert. Never silently drop a notification.
- **Idempotency**: a duplicate event must not send two emails. Key on an event ID and record what has been sent.
- **Delivery status** written back so support can answer "did the customer get it?"

Then land it: *"This is close to what I built — the async part with RabbitMQ for emails, and Kafka for publishing state changes to several consumers."*

Design 2 • — A user onboarding / verification workflow

Closest to your actual domain. If they let you pick an example, pick this.

Clarify: What are the verification steps, and are any of them external? Who reviews a case that fails automated checks? Does it need an audit trail? Can a user resume a half-finished application?

Data model:

```

onboarding_case
  id, user_id, status, current_step, created_at, updated_at

verification_check
  id, onboarding_case_id, check_type, vendor_ref, outcome, checked_at

review_decision
  id, onboarding_case_id, reviewer_id, decision, comments, decided_at

audit_log
  id, entity_type, entity_id, action, actor, before_state, after_state, timestamp

```

The state machine — draw it, it always impresses:

```

DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED → COMPLETED
                        |
                        └──→ REJECTED

```

The points to make:

- **Model the states explicitly** and validate transitions in one place. An approved request must never go back to draft. (*Your resume mentions unit tests for state-transition validation — say that here.*)
- **Authorisation server side.** A requester must not be able to approve their own request. Enforce in the service layer, not the UI.
- **Append-only audit log.** In banking every state change must be attributable — who, what, when, and what it was before. Never update audit rows.
- **Publish each state change as an event** for compliance and reporting consumers. That is your Kafka work.
- **Idempotency on submit**, so a double-click does not create two cases — or, on the payments side, two debits from one intent. This is the single most important rule on a financial platform.

Design 3 — A URL shortener

The classic warm-up. Know it in case it comes up.

Core: `POST /shorten` takes a long URL, returns a short code. `GET /{code}` returns a **301 or 302 redirect**.

Generating the code: a base62 encoding of an auto-increment ID is simple and collision-free. Hashing the URL is an alternative but needs collision handling.

Scale points: it is extremely read-heavy, so cache the code-to-URL mapping in Redis in front of the database. The lookup must be a primary key or unique index hit. Mention 301 (permanent, browser caches it, so you lose click analytics) versus 302 (temporary, every click reaches you) — that trade-off is the interesting part.

Design 4 — "How would you design a REST API for X?"

The generic answer, which is worth having ready:

GET	/api/v1/transactions	list, with pagination and filters
GET	/api/v1/transactions/{id}	one
POST	/api/v1/transactions	create → 201 + Location header
PUT	/api/v1/transactions/{id}	full replace
PATCH	/api/v1/transactions/{id}	partial update
DELETE	/api/v1/transactions/{id}	remove → 204

The rules to state:

- **Nouns, not verbs.** `/accounts/{id}/withdrawals` as a sub-resource, not `/doWithdrawal?id=`.
- **Correct status codes.** 201 for created, 204 for no content, 400 for bad input, 401 vs 403, 404, 409 for a conflict, 422 for validation, 500 for server errors.
- **Pagination on every list endpoint.** Never return an unbounded collection.
- **Versioning** in the URL, and only additive changes within a version.
- **A consistent error shape** across every endpoint, via `@RestControllerAdvice`.
- **Idempotency** for non-safe operations, usually via a client-supplied key.
- **DTOs, never entities, at the boundary.** Returning entities leaks your schema, causes lazy-loading surprises during serialisation, and means a database change breaks your API contract.

That last point is a genuinely good one and few candidates say it.

Design 5 — "This API is slow. What do you do?"

You have lived this. Give a process, not a list of fixes.

1. MEASURE Where is the time actually **going**? Application logs and timings first, **then** the slow query log. Do not guess.
2. CATEGORISE Is it the **data**base, the application, or an external call?
 - **Database** → $N+1$, missing index, bad JOIN
 - **Application** → inefficient loop, unnecessary object creation
 - **External** → a synchronous call that should be async
3. FIX THE BIGGEST **ONE** FIRST
4. RE-MEASURE Prove it. Compare before and after.

Then make it concrete: *"That is exactly the process that took the report from 15 seconds to 6. The measurement step told me it was $N+1$ rather than the query itself, which I would not have guessed."*

The honest line that scores: *"My first assumption about the cause was wrong, which is why I measure before optimising."*

✓ Check yourself before moving on

1. Recite the six-step framework.
2. Design the notification service out loud in 3 minutes, including retries and DLQ.
3. Draw the onboarding state machine and say why the audit log is append-only.
4. Give five REST API design rules, including DTOs at the boundary.

11 · Talking About Your Experience

Time needed: 60 minutes, said out loud

| First, the context: your real position

You have **1 year 8 months** at Zensar on **Tradu**, a trading platform for FXCM, covering user onboarding, identity verification and transaction processing across multiple payment vendors. Java and Spring Boot backend, React frontend, deployed on AWS.

What is genuinely strong here:

- **The domain transfers directly.** Lloyds is a bank. You have worked on KYC-style identity verification and money movement through payment vendors. Most candidates at your level have worked on something generic.
- **You worked across the full development cycle**, not just coding: unit tests with JUnit 5 and Mockito, defect verification, root cause analysis, refactoring, logging, code quality through SonarQube, and Agile delivery in JIRA. That is the profile a bank wants.
- **You collaborated with a frontend team**, so you have had to think about the API as a contract someone else consumes — which is exactly what "API design" questions are about.

What to be careful about: at 1 year 8 months, your credibility comes from being precisely calibrated. **Claim what you did, name where your depth ends, and let the fundamentals carry the rest.** Section 5 covers the four places your resume invites a follow-up you need to be ready for.

| • "Tell me about yourself" — your script

Structure: **Now → What you built → Why this role.** Under two minutes.

"I am a Java backend developer with a bit under two years of experience, most recently at Zensar working on Tradu, a trading platform for FXCM.

It is a Java and Spring Boot backend with a React frontend, deployed on AWS, and it covers user onboarding, identity verification and transaction processing across several payment vendors. My work was the backend business components and REST APIs behind those flows — the controllers and services, the validation and exception handling, and the API security with Spring Security. I wrote the JUnit and Mockito tests for what I built, and I spent a fair amount of time on defect analysis and refactoring, including improving the logging so issues were faster to diagnose.

What draws me to this role is that it is the same domain and the same stack, but on a product being built and run long term rather than a client project. I want to go deeper on Spring Boot and on distributed systems, and do it somewhere the quality bar is high — which in a bank it has to be."

Practise this twice. Time it. **End on why this role**, not on a list of technologies.

| Your stories — build these from what you actually did

For each: **what it was** → **what you decided** → **what was hard** → **what you would do differently**. That last one is what makes an answer sound senior rather than junior.

⚠ **Fill in the specifics yourself.** These are frames, not scripts. An invented detail falls apart on the second follow-up and costs you the room.

Story 1 ● — An API you built end to end (*lead with this*)

The frame:

"One of the flows I owned was [onboarding / a payment flow / a transaction endpoint]. The work was the controller and service layer, the validation on the way in, and the exception handling so the frontend got a consistent error contract rather than stack traces.

The part that needed thought was [the real decision — where validation belonged, how to represent state, how to handle a vendor's error responses]. I went with [X] because [Y].

I wrote JUnit and Mockito tests over the service logic, particularly [the case that mattered], because that is the part that would be expensive to get wrong."

Be ready for: *what did you validate and where, what did the error response look like, what did you mock and why.*

Story 2 ● — Working with external payment vendors

This is your most interesting story for a bank, because it is about failure.

"The platform integrated multiple payment vendors, so a lot of the work was dealing with systems we did not control. Different response shapes, different error semantics, and calls that can time out or hang.

[What you actually did — mapped vendor responses to a common internal model / handled specific failure codes / added logging around the boundary so failures were diagnosable.]"

Then show you understand the general problem, which is the part that scores:

"The thing I have taken from it is that any call to an external system needs a timeout, and it needs to be idempotent or guarded, because on a payment you cannot afford to retry blindly and debit someone twice. Beyond that a circuit breaker stops a vendor that has gone slow from taking your own service down with it."

Story 3 — A defect you root-caused

Your resume says root cause analysis, so expect to be asked for one.

Use STAR and land on **what changed afterwards**: a test added, a log line added, a check added. That turns a firefighting story into a quality story — which is what a bank is listening for.

Story 4 — Refactoring and logging

"I refactored existing code for readability and maintainability, and improved the logging so problems were faster to diagnose. The logging one mattered more than it sounds — when a transaction fails in production you need to be able to follow what happened without attaching a debugger."

If they push on logging, the good detail is: log at the boundaries, include an identifier you can trace a request by, never log card numbers, credentials or personal data — and in a regulated environment that last one is a compliance matter, not a style preference.

●● Section 5 — The four questions your resume invites

Prepare all four. None is a problem; all are a problem if you improvise them.

1. ●● "How much microservices experience do you have?"

Your resume lists microservices; your project description is a Spring Boot backend. **The full answer is in file 05, Q7** — read it properly. The short version:

Name your level first — REST services within a Spring Boot backend, working across service boundaries and with external vendor APIs — say plainly that you have not owned a multi-service architecture end to end, then demonstrate that you understand the model and its **costs**. Honesty plus understanding beats a claim that unravels.

2. ● "You have Kafka, AWS, Docker and Jenkins listed. How deep is that?"

Your resume already says "Apache Kafka fundamentals" and "exposure to" for the cloud and DevOps tools. **That wording is good — hold the line on it in the room.** The trap is being flattered into overclaiming when an interviewer sounds interested.

"I would call that exposure rather than depth. I have used Jenkins and SonarQube as part of our pipeline and Docker in the development setup, and the application was deployed on AWS, but I was not the person owning the infrastructure. With Kafka I know the model — topics, partitions, consumer groups, and why you would publish an event instead of making a synchronous call — but I have not run it in production."

Then, if it is a Kafka conversation, go to file 06 and show the understanding. **Saying "I know the model but have not run it in production" is a completely respectable answer at your level.** Claiming production Kafka and then being asked about consumer group rebalancing is not.

3. • Your employment ended in July 2026

Your resume shows **Dec 2024 – Jul 2026**, and it is now September. That is roughly two months, which is short and recent — but it will be asked, because interviewers always ask.

Two or three sentences, factual, no apology:

"[The real reason — the project ended / the client engagement wound down / I chose to move on.] Since then I have been [what you have actually been doing — interviewing, strengthening Spring Boot and microservices, building something]. I have been looking for a product role rather than taking the next client project."

Fill in the truth. A two-month gap between roles is completely unremarkable. The only thing that makes it a problem is sounding evasive about it.

If you have been learning, be specific. "I have been working through Spring Boot and distributed systems, and built [X]" is worth far more than "I have been upskilling".

4. • The AI tools on your resume

You have listed ChatGPT, GitHub Copilot and Claude under skills and in your experience:

"Used ChatGPT, GitHub Copilot, and Claude AI for debugging, code review, and documentation."

Expect a question about it, and expect it to have an edge, because at a bank there are two real concerns behind it: can you work without the tools, and do you understand what you may and may not paste into them.

The answer that lands:

"I use them the way I would use a senior colleague to think out loud with — for a first draft, for explaining an unfamiliar piece of code, or for spotting something in review. But I do not commit anything I could not explain and defend myself, because I am accountable for it either way, and generated code is confidently wrong often enough that you have to read it properly.

The other part is data. In a regulated environment you cannot paste proprietary code, customer data or credentials into a public tool. I would follow whatever the organisation's policy is on approved tooling — and I would want to know what that policy is on day one."

That second paragraph is the one that matters. It tells a bank you understand the risk without being asked, which is exactly the instinct they are screening for.

Worth considering before the interview: the line about AI tools is unusual on a resume and it invites this question. It is not wrong to have it there — used well it shows modern working — but decide deliberately whether you want to keep it, because it will be asked about. If you keep it, have the answer above ready.

✓ Before the interview

1. Say the "tell me about yourself" script out loud, twice. Time it.
2. Fill in Stories 1 and 2 with **real specifics** from Tradu, and say them out loud.
3. Rehearse all four answers in section 5 — especially the microservices one.
4. Write down the **real one-sentence reason** the role ended in July.
5. Have your GitHub and anything you have built since ready to show.

12 · Behavioural Round, Questions to Ask, Logistics

Time needed: 45 minutes. Say every answer out loud.

| First, the context: why this round is heavier than you expect

UK banks typically run the behavioural round as a **structured competency interview** — prepared questions, asked the same way to every candidate, scored against a framework. That is different from the informal "tell me about yourself" chat common in Indian services interviews.

What that means practically:

- **Prepared examples matter.** "I would generally..." scores badly. "On Tradu, we had..." scores well. They are listening for a specific situation, not a philosophy.
- **Every answer needs a result.** A story that ends at what you did, without what happened, reads as incomplete to someone scoring against a rubric.
- **You will get follow-ups on the same story** — "what would you do differently", "how did the other person react". Pick examples you remember properly, not the most impressive one.

Use STAR: Situation → Task → Action → Result. Under 90 seconds each.

⚠ I cannot confirm Lloyds' exact framework or round structure. Ask your recruiter what the behavioural round covers — it is a normal question and they usually tell you.

| The seven questions to prepare

Have a **specific, real** example for each. Two of them can share a story; the rest should not.

1. • "Tell me about a technical problem you solved."

Use a defect you root-caused. Land on **what changed afterwards** — a test, a log line, a check — so it becomes a quality story rather than a firefighting one.

2. • "Tell me about a time you disagreed with someone."

The pattern that scores: you raised the concern, then brought **evidence or an alternative** rather than just objecting, and you accepted the outcome either way.

"In a review I flagged that [X]. Rather than just saying it was wrong, I [showed the case where it broke / wrote a quick test demonstrating it]. We [outcome]. What I have learned is that showing beats arguing — it turns an opinion into something we can both look at."

3. • "Tell me about a mistake you made."

Have one. Refusing to answer looks far worse than the mistake. Real, owned plainly, ending in what you changed.

A missed edge case caught in QA, with "I added a test for that case and started checking the negative paths in review", is a good shape. Do not pick something that suggests carelessness with money or data.

4. "How do you handle pressure or a tight deadline?"

Scope, communicate early, never silently slip. You worked in sprints with QA — you know what happens when something half-finished reaches testing.

5. "Tell me about working with someone difficult, or in a team."

Your frontend collaboration is good material: two people with different priorities agreeing an API contract. The useful point is **agreeing the contract early** rather than discovering the mismatch in integration.

6. "How do you learn something new?"

"Documentation first, because tutorials teach you one path and the docs teach you the model. Then I build something small, because reading does not surface the problems. Then I look at how it is already used in the codebase, since the team's conventions matter more than my preferences."

7. • "Why do you want to work at Lloyds?"

Do not say "it is a big company" or "for growth".

"Two reasons that are specific to me. First, the domain — I have spent nearly two years on a trading platform doing onboarding, identity verification and payments across multiple vendors, so financial services is where my experience actually is, and I would rather build on that than start again somewhere unrelated.

Second, I want to work on a product that is built and run over the long term rather than a client project I hand over. That changes how you write things — you care about tests, logging and maintainability differently when you are the one who will be supporting it in two years. The quality bar being high in a regulated environment is a reason I want it, not a reason to be nervous about it."

That answer is strong because it is true and because almost nobody says the second part.

| Two questions to expect about your situation

"Why are you leaving / why did the role end?" → See file 11, section 5.3. Short, factual, no criticism of the employer. Frame as moving **toward** something.

"You have under two years. Why are you ready for this?"

"I would not claim to be a senior engineer. What I would say is that the two years were dense — I worked across the full cycle rather than just writing code: tests, defect analysis, refactoring, code review and releases, on a platform handling real money. I am looking for somewhere I can go deeper with people who are further along, and I would rather be the one asking questions in a strong team than the most experienced person in a weak one."

• Questions YOU ask them

Have four ready, ask two or three. Ending with "no, I'm good" reads as no interest.

Technical:

1. "What does the service I would be working on actually do, and who uses it?"
2. "What does the stack look like day to day — Spring Boot version, cloud platform, how you deploy?"
3. "How much of the work is new build versus modernising existing systems?"
4. "What does the testing and review process look like before something reaches production?"

Team and growth:

5. "What would a successful first three months look like for someone joining at my level?"
6. "Who would I be learning from, and how does code review work in the team?"
7. "How does the team handle production support — is there an on-call rota?"

The one that makes you memorable:

"What is the biggest technical problem the team is working on right now?"

Then **listen**, and connect your answer to it. That is how you close an interview.

Logistics

Question	How to answer
Notice period	You are not currently employed, so you are immediately available . Say it clearly — it is a genuine advantage over candidates serving 60 or 90 days.
Location / relocation	Know whether the role is Hyderabad-based and whether you can be there. Answer directly; do not hedge.
Work from office	Most banks are largely back in office. Answer honestly.
Shift overlap	A UK bank means some overlap with UK hours. You worked with FXCM, so say that — you know what it involves.

Question	How to answer
Salary	Know your last fixed CTC and your target before the call. Never inflate your previous CTC; it is verified against payslips. Ask for the fixed/variable split at offer stage.
Other offers	Honest and brief. Do not invent leverage.

| On the day

- Printed resume, ID, laptop charged, charger, backup hotspot, water.
- **Know your own resume cold.** Everything on it is fair game — Spring Security, Hibernate, JUnit, Mockito, SonarQube, Jenkins, Docker, AWS, Kafka, React, **and the AI tools line.** If you touched something briefly, say so rather than overstating.
- If online: join five minutes early, test camera and microphone, plain quiet background.
- Smart dress. Slightly overdressed beats underdressed.

| If you get stuck

- **Do not freeze — narrate.** "Let me think about that for a second" is a complete sentence.
- **Do not bluff.** "I have not used that directly. My understanding is X — is that the direction you mean?" has never cost anyone an offer. Bluffing has.
- **Ask what they are getting at.** "Do you mean at the database level or in the application?"
- If you fumble one question, let it go. People lose interviews by replaying question 4 while answering question 7.

13 · Testing, Code Review and Engineering Practice

Time needed: 60 minutes

This file is where you can look better than your years. Most candidates at 1-2 years prepare Java and Spring and nothing else. Your resume leads with JUnit 5, Mockito, SonarQube, code review, logging and root cause analysis — and a bank cares about all of it, because in a regulated environment the *way* code gets built is part of the product.

Part A · Testing

| Q1. ●● Unit test vs integration test — and what you actually test

Say this.

"A unit test exercises one class in isolation with its collaborators mocked, so it is fast and tells you exactly what broke. An integration test wires real components together — the repository against a database, or the controller through the full Spring context — and tells you that the pieces actually fit.

I write mostly unit tests over service logic, because that is where the business rules live, plus a smaller number of integration tests over the paths that would be expensive to get wrong. What I aim to test is **behaviour and rules**, not lines — the negative cases especially, because those are the ones that cost money."

Real-time example. On a payment flow the valuable tests are not "a successful payment succeeds". They are: a payment below the minimum is rejected, a duplicate request does not debit twice, a vendor timeout leaves the transaction in a recoverable state rather than an unknown one. Those are the rules; the happy path mostly tests itself in QA.

Q2. ●● Mockito — what you mock and why

```
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {

    @Mock PaymentVendorClient vendorClient;    // external - always mock
    @Mock TransactionRepository repository;    // database - mock in a unit test
    @InjectMocks PaymentService service;      // the thing under test

    @Test
    void marksTransactionFailedWhenVendorRejects() {
        when(vendorClient.initiate(any())).thenReturn(VendorResponse.rejected("INSUFFICIENT"));

        service.process(new PaymentRequest("acc-1", new BigDecimal("100.00")));

        ArgumentCaptor<Transaction> saved = ArgumentCaptor.forClass(Transaction.class);
        verify(repository).save(saved.capture());
        assertThat(saved.getValue().getStatus()).isEqualTo(FAILED);
        verifyNoMoreInteractions(vendorClient);
    }
}
```

Say this.

"I mock the things I do not own or that make the test slow and non-deterministic — external vendor clients, repositories, anything over the network. I do not mock the class under test, and I try not to mock value objects, because then the test is just asserting my own mock setup.

`when(...).thenReturn(...)` sets up the input; `verify(...)` checks an interaction actually happened, which matters when the important outcome is a side effect rather than a return value. `ArgumentCaptor` is what I use when I need to assert on *what* was passed, not just that something was."

Know these terms: `@Mock` creates the double, `@InjectMocks` builds the class under test with the mocks injected, `@Spy` wraps a real object so most calls are real, and `@MockBean` is the Spring version that replaces a bean in the application context.

The trap question — "what is the difference between a mock and a stub?"

"A stub just returns canned answers so the test can proceed. A mock also records how it was called, so you can verify the interaction. Mockito does both, which is why people use the words loosely."

Q3. "What is a good test?" and the coverage question

Say this.

"A good test fails for exactly one reason, and the name tells you what broke without reading the body. I name them as behaviour — `marksTransactionFailedWhenVendorRejects` — rather than `testProcess1` .

On coverage: it tells you what was *executed*, not what was *verified*. A test with no assertion still counts towards it. So I would rather have the critical paths genuinely covered than chase a number — though a very low number is still a useful warning sign, and a gate in the pipeline is a reasonable safety net."

Arrange, Act, Assert is worth naming as the structure you write them in.

Q4. TDD — have an honest opinion

"I have not worked strictly test-first, so I would not claim to be a TDD practitioner. What I do find genuinely useful is writing the test for a bug **before** fixing it, because that proves you have actually reproduced it and it stops the bug coming back."

That is honest, it is true of almost everyone, and the second sentence shows you have thought about why the practice exists.

Part B · Code review and code quality

Q5. • "What do you look for in a code review?"

Say this — in this order, because the order is the answer.

"First **correctness**: does it do what the ticket says, and are the edge cases handled — nulls, empty results, boundaries. Second **error handling**: is anything being swallowed silently, which is the worst outcome because the failure becomes invisible.

Then the **data layer**, because that is where the expensive problems hide — a query inside a loop, a missing index, `@Transactional` on the wrong method. Then **tests** — not the count, whether the meaningful cases are covered. Then **security** — is input validated, is authorisation checked server side, is anything sensitive being logged. And **readability** last: naming, method length, duplication. It matters, but I would not block a change on it alone."

On giving feedback:

"I separate what must change from what is a suggestion and say which is which, because 'this could be better' just wastes the author's time. I ask rather than instruct when I do not know the context. And if a thread goes past three replies I go and talk to them, because that usually means we are misunderstanding each other rather than disagreeing."

Q6. SonarQube and static analysis

Say this.

"SonarQube runs static analysis in the pipeline and reports bugs, vulnerabilities, code smells, duplication and coverage. The value is that it catches the mechanical things — an unclosed resource, a possible null dereference, a hardcoded credential — so that human review can spend its time on design and correctness instead.

The thing to be careful about is treating every finding as equally important. A genuine vulnerability and a slightly long method are both 'issues' in the report and are not remotely the same, so it needs judgement rather than driving the count to zero."

Quality gate is the term worth knowing: a threshold in the pipeline that fails the build if new code drops below the standard — often applied to *new* code only, so an existing codebase can improve without blocking everything.

Q7. Logging and observability

Say this.

"I improved the logging on Tradu specifically so issues were faster to diagnose. The principles I would apply: log at the boundaries — incoming request, outgoing vendor call, and the result — so you can see where a flow stopped. Include an identifier that lets you trace one transaction through the logs. Use levels properly, so ERROR means something needs attention rather than being noise.

And **never log sensitive data** — card numbers, credentials, personal details. In a regulated environment that is a compliance breach, not a style problem. That is the part I would check in review."

If asked about production support: logs first, then what changed recently, then reproduce. Form a hypothesis and confirm it before fixing, because fixing the wrong thing hides the real problem.

Part C · Delivery

Q8. Git and branching

Say this.

"Feature branches off the main line, small commits with messages that say why rather than what, and a pull request reviewed before merge. I rebase my own branch to keep history readable but I would not rewrite history on a shared branch. Merge conflicts I resolve by understanding both changes rather than picking a side — in business logic, taking one blindly silently drops someone's work."

Q9. CI/CD, honestly at your level

Say this.

"I worked within a Jenkins pipeline rather than owning it. What I understand is the shape: a commit triggers a build, the unit tests run, SonarQube analysis runs against the quality gate, and an artefact is produced and deployed to an environment. The point is that the same artefact is promoted rather than rebuilt per environment, so what was tested is what ships.

I would not claim to have built pipelines from scratch, but I am comfortable working in one and fixing my own build failures."

Calibrated honesty. Do not claim to have built the pipeline.

Q10. Agile in practice

"Two-week sprints, stand-ups, JIRA for tracking, and refinement where we sized the work. The part I found most useful was refinement, because that is where ambiguity in a ticket gets caught — before someone has built the wrong thing."

If asked about estimation: break it down until you can picture doing each piece, include testing and review time, and **state your assumptions** so a changed assumption is a re-estimate rather than a missed deadline.

Q11. • Secure coding — worth having for a bank

Even a short answer here is unusual at your level and it lands well.

"The basics I would always check: validate input at the boundary and never trust the client; use parameterised queries or JPA rather than string-concatenated SQL, which is how injection happens; enforce authorisation server side on every endpoint, not in the UI; never log or expose sensitive data; and keep secrets out of code and configuration in the repository — environment variables or a secrets manager.

And return generic error messages to the client while logging the detail internally, because a stack trace tells an attacker about your framework versions and internal structure."

✓ Check yourself before moving on

1. What you mock and what you do not, and why.
2. Write the Mockito test above from memory — `@Mock`, `@InjectMocks`, `when`, `verify`.
3. What you look for in a code review, **in order**.
4. The coverage answer — what it measures and what it does not.

5. Three secure coding practices, without prompting.

14 · Final Cheat Sheet — INTERVIEW DAY ONLY

45 minutes. Nothing else. No new material.

If a line here does not ring a bell, open that file for three minutes and come back.

The 8 answers that carry this interview

1. **HashMap:** "`hashCode()` , then a spread of $h \wedge (h \ggg 16)$, then bucket index $(n-1) \& hash$. Collisions chain in a linked list, and since Java 8 a bucket over 8 entries becomes a red-black tree. Default capacity 16, load factor 0.75, so it resizes and rehashes at 12."
 2. **equals/hashCode:** "Equal objects must have equal hash codes. Override `equals` without `hashCode` and the map looks in the wrong bucket and never finds the entry."
 3. **N+1:** "One query for the list, then one per row for the relation. Find it by counting queries in the SQL log. Fix with JOIN FETCH, `@EntityGraph` , batch fetching, or a DTO projection."
 4. **@Transactional internal call:** "It works through a proxy. An internal `this.method()` call bypasses the proxy, so no transaction starts." Rolls back on **unchecked** exceptions only.
 5. **Constructor injection:** "Fields can be final, dependencies cannot be missing, and I can test with `new Service(mock)` without a Spring context."
 6. **Microservices trade:** "You buy independent deployment, scaling and fault isolation, and you pay with network failure, no distributed transaction and harder debugging." **Say the cost — that is the part that scores.**
 7. **Mocking:** "I mock what I do not own or what makes the test slow — vendor clients, repositories. Not the class under test."
 8. **Index cost:** "Faster reads, but every insert and update maintains every index, so I add them deliberately."
-

• The four honesty answers — rehearse these most

Microservices depth → "On Trudu I built and consumed REST services inside a Spring Boot backend and worked with external vendor APIs. I have not owned a multi-service architecture end to end." **Then immediately:** the trade-off in answer 6 above. Name your level first — it buys credibility for everything else.

Kafka / AWS / Docker / Jenkins → "Exposure rather than depth. I know the Kafka model — topics, partitions, consumer groups, why you publish an event instead of calling synchronously — but I have not run it in production." **Do not get flattered into overclaiming.**

Role ended July 2026 → Two sentences, the real reason, what you have been doing since. No apology. Two months is unremarkable; evasiveness is not.

The AI tools line → "I use them to think out loud with, but I do not commit anything I could not explain and defend myself. And in a regulated environment you cannot paste proprietary code or customer data into a public tool — I would follow the organisation's policy on approved tooling." **The data point is the one that matters.**

Core Java

- **Overloading** = compile time, different parameters. **Overriding** = runtime, subclass. You cannot override a static method — it is hidden.
- **Abstract class** = shared state and code, one only. **Interface** = capability contract, many. *Payment vendors = interface. Base transaction = abstract class.*
- **ArrayList** O(1) get · **LinkedList** O(n) get. Default to ArrayList.
- **HashMap** not thread safe · **Hashtable** locks the whole map, legacy · **ConcurrentHashMap** locks per bucket.
- **String immutable** — string pool, security, thread safety, cached hash.
- `==` references, `.equals()` values. `new String("a") != "a"`.
- **Checked** = compiler forces handling · **unchecked** extends `RuntimeException`.
- `finally` always runs except `System.exit()`. Never `return` from `finally`.

Java 8

- **Intermediate** ops lazy (`filter` , `map` , `sorted`) · **terminal** trigger (`collect` , `reduce` , `findFirst`).
- `map` one-to-one · `flatMap` one-to-many then flattened.
- `Predicate` →filter · `Function` →map · `Consumer` →forEach · `Supplier` →lazy.
- `groupingBy(X, counting())` · `groupingBy(X, summingDouble(Y))`.
- **Optional**: `map` / `orElse` / `orElseThrow` , not `isPresent` + `get` .

Spring Boot

- **IoC** = framework controls creation · **DI** = how it hands dependencies in.
- Beans are **singleton by default** → keep them **stateless**. A mutable field is a concurrency bug and, on a financial platform, an audit failure.
- `@Repository` also **translates database exceptions**.
- `@RestControllerAdvice` = one error shape. Log the detail, return a generic message.
- **Auto-configuration** = `@ConditionalOnClass` + `@ConditionalOnMissingBean` . Your bean wins.
- `@WebMvcTest` slice · `@SpringBootTest` full · `@MockBean` replaces a bean.

JPA and Hibernate

- **JPA** spec · **Hibernate** implementation · **Spring Data JPA** the layer on top.
- `@ManyToOne` / `@OneToOne` default **EAGER** — set them **LAZY**. `@OneToMany` / `@ManyToMany` **LAZY**.
- **Dirty checking** — a managed entity needs no `save()`.
- `@Enumerated(EnumType.STRING)`, never **ORDINAL**. Money is **BigDecimal**, never `double`.
- **REQUIRES_NEW** = the audit row survives when the main transaction rolls back.

Microservices

- **Discovery** so addresses are not hard-coded · **API gateway** = one entry point, auth and routing in one place · **database per service** = no joins, no distributed transaction.
- **Circuit breaker**: closed → open → half-open. Failing fast protects both sides.
A payment vendor that hangs must not take your service down.
- **Always a timeout**. Only retry **idempotent** operations — never a blind payment retry.
- **Saga** = local transactions + **compensating** transactions. Eventual consistency.
- Split by **business capability**, not technical layer. **Strangler fig**, never big-bang.

Security

- **JWT** = header.payload.signature. **Signed, not encrypted** — nothing sensitive in it.
- Stateless, so any instance validates it — that is why it suits microservices.
- Authorisation **server side**. A UI check only hides the button.
- **BCrypt** — slow and salted by design.
- Parameterised queries, not concatenated SQL. Secrets out of the repo.

SQL

- **Left-most prefix**: an index on `(status, created_at)` does not serve `created_at` alone.
- Index killers: a **function on the column**, a **leading %** in LIKE.
- **EXPLAIN**: `type: ALL` = full scan. Check `key`, `rows`, `Extra`.
- **WHERE** before grouping · **HAVING** after.
- Deep **OFFSET** is slow → keyset pagination.

Testing and quality • (your strong ground)

- **Unit** = isolated, mocked, fast · **integration** = wired together, proves the pieces fit.
- **Mock** what you do not own or what is slow. `@Mock` + `@InjectMocks` ·
`when().thenReturn()` sets up · `verify()` checks a side effect · `ArgumentCaptor` asserts on what was passed.
- Test **behaviour and rules**, especially the **negative** cases — duplicate request does not debit twice, below-minimum is rejected.
- **Coverage** measures what ran, not what was verified.

- **Code review order:** correctness → error handling → data layer → tests → security → readability last.
 - **Logging:** at the boundaries, with a traceable id, correct levels, **never sensitive data**.
 - **SonarQube:** catches the mechanical issues so review can spend its time on design. Quality gate on new code.
-

| Your stories

Tradu (FXCM) — trading platform: onboarding, identity verification, transaction processing across multiple payment vendors. Java/Spring Boot backend, React frontend, AWS.

What you did: backend business components, controllers and services · REST APIs consumed by React · Spring Security, validation, exception handling · JUnit 5 and Mockito · refactoring · improved logging · defect verification and root cause analysis · Jenkins, SonarQube, Postman, Swagger.

The vendor story is your best one — external systems you do not control, so timeouts, idempotency and not debiting someone twice.

| Opening and closing

"Tell me about yourself" → Now → What you built → **Why this role**. Under 2 minutes.

"Why Lloyds?" → Domain fit — nearly two years on a trading platform doing onboarding, verification and payments. Plus: *"a product built and run long term changes how you write things — you care about tests, logging and maintainability differently when you will be supporting it in two years."*

Your closing question → *"What is the biggest technical problem the team is working on right now?"*

| The last five minutes

- **You are immediately available.** Most candidates are serving 60–90 days. Say it.
- **Name your level before they ask.** It makes everything else you say credible.
- **If you do not know something: say so, then reason out loud.** That has never cost anyone an offer. Bluffing has.
- Give the **why**, not just the **what**.
- Sit up. Breathe out slowly. Smile before you speak — it changes your voice.

Go get it.